

Sarafu Network Protocol

Swap pool, oracle pricing, vouchers, distribution and registry contracts

Project	Sarafu Network Protocol (c1c-protocol)
Commit	f0944d9
Scope	all 17 contracts in src/, plus cmd/ge-publish where it determines on-chain configuration
Reviewed first	SwapPool, OracleQuoter, SwapRouter — the swap path, and the largest share of findings
Compiler	solc 0.8.34, evm_version = osaka, optimizer on (200 runs)
Date	2026-08-07

Findings: 75

Critical	High	Medium	Low	Info
2	5	21	31	16

Scope and limitations

This assessment covers every contract in `src/` at the commit named on the cover, together with `cmd/ge-publish` where it determines on-chain configuration, and `docs/` where documented procedure affects the deployed security posture.

Findings are ordered by severity. Each carries the file and line range it applies to, the mechanism by which it fails, a concrete reproduction, and a specific recommendation. Every finding is reproduced by an executing test in `test/audit/`; the test names are cited in each finding so a reader can verify a claim rather than take it on trust.

Severity reflects impact and the conditions required to reach it. *Critical* and *High* denote loss or freezing of funds, or loss of a security control, under conditions the code permits. *Medium* denotes value leakage, denial of service, or a violated invariant that integrations rely on. *Low* and *Informational* denote defects with bounded or indirect consequence, and deviations between code and specification.

What this review does not establish. The absence of a finding is not evidence of correctness. Coverage of exploit classes, and the classes and contracts that did *not* receive systematic treatment, are stated explicitly in *Method*, *surface coverage*, and *what was not done*. Economic and governance design are out of scope: the review assesses whether the code does what the specification says, not whether the specification is a good idea.

LLM-generated. Derived from `audit/AUDIT.md`, which remains the authoritative version.

Contents

Contents

1	Summary	1
2	Critical severity	5
	C-1 — OracleQuoter multiplier above parity drains the pool	5
	C-2 — <code>_deposit</code> credits the requested amount, not the amount received	6
3	High severity	7
	H-1 — Swaps accept no <code>minAmountOut</code> or deadline	7
	H-2 — The seal mechanism provides no durable guarantee	7
	H-3 — <code>Splitter</code> 's allocation sum wraps in <code>uint32</code> , defeating its only arithmetic invariant	9
	H-4 — <code>EthFaucet</code> fails open, and the documented deployment ships an unlimited public ETH dispenser . . .	10
	H-5 — <code>EthFaucet.seal()</code> can permanently lock gating that was never configured	10
4	Medium severity	12
	M-1 — <code>reverseValueFor</code> inverts the rounding stages in the wrong order	12
	M-2 — <code>fees[]</code> credited after the outbound transfers permits reentrant fee inflation	13
	M-3 — <code>tokenOut</code> is never checked against <code>tokenRegistry</code>	14
	M-4 — A single global <code>maxStaleness</code> cannot serve feeds with different heartbeats	14
	M-5 — The fee domain is a joint constraint nobody enforces, and at its boundary a swap pays out zero . . .	15
	M-6 — <code>withdrawLiquidity</code> ignores <code>fees[]</code> , taking reserved fees and bricking the token	16
	M-7 — Permissionless <code>deposit</code> plus a total-balance cap is a cheap denial of service	16
	M-8 — <code>RelativeQuoter</code> divides before multiplying, breaking the same round-trip guarantee as M-1	17
	M-9 — <code>AccountsIndex.deactivate()</code> does not revoke the whitelist it appears to control	18
	M-10 — An empty token symbol creates a permanent, unremovable ghost entry	18
	M-11 — <code>TokenUniqueSymbolIndex</code> de-duplicates symbols but not tokens, permanently squatting symbol keys .	19
	M-12 — <code>Splitter</code> accepts <code>address(0)</code> and itself as recipients	20
	M-13 — One hostile or broken recipient blocks <code>Splitter</code> distribution for everyone	20
	M-14 — Permissionless distribution plus "remainder to the last recipient" lets that recipient capture a dripped stream	21
	M-15 — <code>Splitter.updateSplit</code> retroactively redirects funds already deposited	21
	M-16 — <code>EthFaucet</code> has no <code>withdraw</code> path, so ETH can be locked permanently	22
	M-17 — <code>giveTo</code> rate-limits the recipient, not the caller, so recipient rotation defeats the cooldown	22
	M-18 — The faucet's registry and period calls never validate <code>returndata</code>	22
	M-19 — <code>GiftableToken</code> 's expired flag and <code>Expired</code> event never persist	23
	M-20 — <code>burn()</code> is blocked after expiry, so an expired token's supply can never be retired	24
	M-21 — Every <code>GiftableToken</code> holder carries an undocumented, unrevocable infinite allowance to <code>Permit2</code> .	24
5	Low severity	26
	L-1 — With <code>feeAddress == address(0)</code> the pool fee is charged but never recorded	26
	L-2 — In coupled mode accrued fees are re-lent as swap liquidity	26

L-3 – Oracle freshness checks are incomplete and fail with an unattributable panic	27
L-4 – getScale’s 77-decimal bound is unreachable	27
L-5 – A quote that truncates to zero consumes the input and returns nothing	28
L-6 – maxStaleness has no fallback for zero, which is what an in-place upgrade reads	29
L-7 – Every proxied contract accepts owner == address(0) and consumes the initializer	29
L-8 – The registry check accepts any over-long reply and cannot report a codeless registry	31
L-9 – protocolFeeController is write-once and unsealable, so an unreceivable recipient bricks every swap	32
L-10 – FeePolicy cannot express a zero fee for a pair, and its event says otherwise	32
L-11 – AccountsIndex.time() leaks the blocked flag into the returned timestamp	33
L-12 – AccountsIndex.add(address(0)) whitelists the null address	34
L-13 – A bytes32-returning symbol() cannot be indexed	34
L-14 – TokenUniqueSymbolIndex.initialize does not enforce equal array lengths	35
L-15 – ContractRegistry.initialize accepts duplicate identifiers	35
L-16 – Splitter validation is $O(n^2)$ and re-runs on every distribution, with no cap on recipients	35
L-17 – Splitter.getHash uses abi.encodePacked on two dynamic arrays; the collision is real and unreachable only by accident	36
L-18 – RescueVault’s payable catch-all fallback reports success for every unknown selector	37
L-19 – RescueVault.admin is immutable with no rotation path, and rotation attempts fake success	37
L-20 – RescueVault batch sweeps are all-or-nothing, and duplicate ids revert them	38
L-21 – GiftableToken allows mint and transfer to address(0)	38
L-22 – Unbounded setPeriod overflows lastUsed + period and selectively bricks returning claimants	38
L-23 – A poke consumes a subject’s cooldown with no payout, and cannot be undone	39
L-24 – amount == 0 still consumes the recipient’s cooldown	39
L-25 – PeriodSimple.next() contradicts have() for a never-poked subject	39
L-26 – nextTime reverts undecodably when unset, and nextBalance ignores its own argument	40
L-27 – The expiry timestamp has no getter, and initialize accepts an expiry already in the past	40
L-28 – EthFaucet.giveTo(address(0)) burns ETH: there is no zero-recipient check	41
L-29 – The limiter cap is checked against the pre-transfer balance, so a hooked tokenIn can nest deposits and exceed it	41
L-30 – ge-publish publish-one --contract swappool rejects every quoter value; the command is dead	42
L-31 – Proxy salts are derived only from (deployer, contract name): one proxy per contract name per deployer per factory	43
6 Informational	45
I-1 – supportsInterface advertises incorrect interface IDs	45
I-2 – Three declared errors in SwapPool are never used	45
I-3 – SwapRouter does not validate path continuity	45
I-4 – Seal state-machine rough edges	45
I-5 – try/catch cannot catch decode failures, so two named errors are unreachable	46
I-6 – isActive() is declared on both integration interfaces and never consulted	47
I-7 – The protocol fee and the net output are unobservable from events	47
I-8 – AccountsIndex.remove() reverts with the inverted error, and a test enshrines it	47
I-9 – Registry hygiene: enumeration asymmetry and no token sanity check	48
I-10 – sweepERC721s skips the zero-recipient check on an empty array	48

I-11 – PeriodSimple’s default period == 0 is a one-second cooldown, not ”no limit”	48
I-12 – CAT preference lists: duplicates, no opt-out, untraceable overrides, unvalidated entries	49
I-13 – Ceiling-add overflow in the reverse quote paths panics where the forward direction succeeds	49
I-14 – A GiftableToken held by a SwapPool is permanently locked at expiry, and its frozen balance keeps counting toward liquidity	50
I-15 – withdrawLiquidity has no zero-address check on to	50
I-16 – Deployment tooling: the private key travels on the command line, and over-long registry identifiers are silently truncated	51
7 Method, surface coverage, and what was not done	52
8 Test coverage gaps	55

Summary

ID	Severity	Title
C-1	Critical	OracleQuoter multiplier above parity makes every swap profitable in both directions, draining the pool
C-2	Critical	_deposit credits the requested amount instead of the amount received
H-1	High	Swaps accept no minAmountOut or deadline
H-2	High	The seal mechanism provides no durable guarantee — it is defeated four independent ways
H-3	High	Splitter's allocation sum wraps in uint32, defeating its only arithmetic invariant
H-4	High	EthFaucet fails open, and the documented deployment ships an unlimited public ETH dispenser
H-5	High	EthFaucet.seal() can permanently lock gating that was never configured
M-1	Medium	reverseValueFor inverts the two rounding stages in the wrong order, breaking the documented IQuoter round-trip guarantee
M-2	Medium	fees[] is credited after the outbound transfers, so a hooked tokenOut can re-enter and permanently brick the pool
M-3	Medium	tokenOut is never checked against tokenRegistry
M-4	Medium	A single global maxStaleness cannot serve feeds with different heartbeats
M-5	Medium	The fee domain is a joint constraint across two contracts that nobody enforces; at its boundary a swap pays out zero
M-6	Medium	withdrawLiquidity ignores fees[], taking reserved fees and permanently bricking the token
M-7	Medium	Permissionless deposit plus a total-balance cap is a cheap input-side denial of service
M-8	Medium	RelativeQuoter divides before multiplying, breaking the same round-trip guarantee as M-1
M-9	Medium	AccountsIndex.deactivate() does not revoke the whitelist it appears to control
M-10	Medium	An empty token symbol creates a permanent, unremovable ghost registry entry
M-11	Medium	TokenUniqueSymbolIndex de-duplicates symbols but not tokens, permanently squatting symbol keys
M-12	Medium	Splitter accepts address(0) and itself as recipients
M-13	Medium	One hostile or broken recipient blocks Splitter distribution for everyone
M-14	Medium	Permissionless distribution plus remainder-to-last lets that recipient capture a dripped stream
M-15	Medium	Splitter.updateSplit retroactively redirects funds already deposited
M-16	Medium	EthFaucet has no withdraw path, so ETH can be locked permanently
M-17	Medium	giveTo rate-limits the recipient, not the caller, so recipient rotation defeats the cooldown
M-18	Medium	The faucet's registry and period calls never validate returndata
M-19	Medium	GiftableToken's expired flag and Expired event never persist
M-20	Medium	burn() is blocked after expiry, so an expired token's supply can never be retired
M-21	Medium	Every GiftableToken holder carries an undocumented, unrevocable infinite allowance to Permit2
L-1	Low	With feeAddress == address(0) the pool fee is charged to the user but never recorded
L-2	Low	In coupled mode accrued fees are re-lent as swap liquidity, so fees[] can exceed the balance
L-3	Low	Oracle freshness checks are incomplete and fail with an unattributable panic
L-4	Low	getScale's 77-decimal bound is unreachable; the surrounding products overflow first
L-5	Low	A quote that truncates to zero consumes the caller's input and returns nothing
L-6	Low	maxStaleness has no fallback for zero, which is what an in-place upgrade reads

ID	Severity	Title
L-7	Low	Every proxied contract accepts <code>owner == address(0)</code> and consumes the initializer
L-8	Low	The registry check accepts any over-long reply and cannot report a codeless registry
L-9	Low	<code>protocolFeeController</code> is write-once and unsealable, so an unreceivable recipient bricks every swap
L-10	Low	<code>FeePolicy</code> cannot express a zero fee for a pair, and its event says otherwise
L-11	Low	<code>AccountsIndex.time()</code> leaks the blocked flag into the returned timestamp
L-12	Low	<code>AccountsIndex.add(address(0))</code> whitelists the null address
L-13	Low	A <code>bytes32</code> -returning <code>symbol()</code> cannot be indexed
L-14	Low	<code>TokenUniqueSymbolIndex.initialize</code> does not enforce equal array lengths
L-15	Low	<code>ContractRegistry.initialize</code> accepts duplicate identifiers
L-16	Low	Splitter validation is $O(n^2)$ and re-runs per distribution, with no recipient cap
L-17	Low	<code>Splitter.getHash</code> collides under <code>abi.encodePacked</code> ; unreachable only by check ordering
L-18	Low	<code>RescueVault</code> 's payable catch-all fallback reports success for every unknown selector
L-19	Low	<code>RescueVault.admin</code> is immutable with no rotation path, and rotation attempts fake success
L-20	Low	<code>RescueVault</code> batch sweeps are all-or-nothing, and duplicate ids revert them
L-21	Low	<code>GiftableToken</code> allows mint and transfer to <code>address(0)</code>
L-22	Low	Unbounded <code>setPeriod</code> overflows <code>lastUsed + period</code> and selectively bricks returning claimants
L-23	Low	A poke consumes a subject's cooldown with no payout, and cannot be undone
L-24	Low	<code>amount == 0</code> still consumes the recipient's cooldown
L-25	Low	<code>PeriodSimple.next()</code> contradicts <code>have()</code> for a never-poked subject
L-26	Low	<code>nextTime</code> reverts undecodably when unset, and <code>nextBalance</code> ignores its own argument
L-27	Low	The expiry timestamp has no getter, and <code>initialize</code> accepts an expiry already in the past
L-28	Low	<code>EthFaucet.giveTo(address(0))</code> burns ETH: there is no zero-recipient check
L-29	Low	The limiter cap is checked against the pre-transfer balance, so a hooked <code>tokenIn</code> can nest deposits and exceed it
L-30	Low	<code>ge-publish publish-one --contract swappool</code> rejects every quoter value; the command is dead
L-31	Low	Proxy salts are derived only from (deployer, contract name): one proxy per contract name per deployer per factory
I-1	Info	<code>supportsInterface</code> advertises incorrect interface IDs
I-2	Info	Three declared errors in <code>SwapPool</code> are never used
I-3	Info	<code>SwapRouter</code> does not validate path continuity
I-4	Info	Seal state-machine rough edges
I-5	Info	<code>try/catch</code> cannot catch decode failures, so <code>OracleCallFailed</code> and <code>TokenCallFailed</code> are unreachable
I-6	Info	<code>isActive()</code> is declared on both integration interfaces and never consulted
I-7	Info	The protocol fee and the net output are unobservable from events
I-8	Info	<code>AccountsIndex.remove()</code> reverts with the inverted error, and a test enshrines it
I-9	Info	Registry hygiene: enumeration asymmetry and no token sanity check
I-10	Info	<code>sweepERC721s</code> skips the zero-recipient check on an empty array

I-11	Info	PeriodSimple's default period == 0 is a one-second cooldown, not "no limit"
I-12	Info	CAT preference lists: duplicates, no opt-out, untraceable overrides, unvalidated entries
I-13	Info	Ceiling-add overflow in the reverse quote paths panics where the forward direction succeeds
I-14	Info	A GiftableToken held by a SwapPool is permanently locked at expiry, and its frozen balance keeps counting toward liquidity
I-15	Info	withdrawLiquidity has no zero-address check on to
I-16	Info	Deployment tooling: the private key travels on the command line, and over-long registry identifiers are silently truncated

Severity	Count
Critical	2
High	5
Medium	21
Low	31
Info	16
Total	75

See [Method](#), [surface coverage](#), and [what was not done](#) for which exploit classes were swept, and which were not.

Findings by contract

Contract	Findings
SwapPool	C-2 H-1 H-2 M-2 M-3 M-5 M-6 M-7 L-1 L-2 L-5 L-7 L-8 L-9 L-29 I-2 I-4 I-6 I-7 I-13 I-14 I-15
OracleQuoter	C-1 M-1 M-4 L-3 L-4 L-6 L-7 I-1 I-5
Splitter	H-3 M-12 M-13 M-14 M-15 L-7 L-16 L-17
EthFaucet	H-4 H-5 M-16 M-17 M-18 L-7 L-24 L-26 L-28
GiftableToken	M-19 M-20 M-21 L-7 L-21 L-27 I-14
TokenUniqueSymbolIndex	M-10 M-11 L-7 L-13 L-14 I-9
PeriodSimple	L-7 L-22 L-23 L-25 I-11
RescueVault	L-18 L-19 L-20 I-10
AccountsIndex	M-9 L-7 L-11 L-12 I-8
RelativeQuoter	M-8 L-7 L-8 I-1 I-5 I-13
FeePolicy	M-5 L-7 L-10 I-6
ProtocolFeeController	M-5 L-7 L-9 I-6
CAT	L-7 I-12
ContractRegistry	L-7 L-15
DecimalQuoter	L-5 I-5
Limiter	L-7
SwapRouter	I-3 — and it is where M-1 and M-8 surface
cmd/ge-publish	H-2 (admin := owner) L-14 (length guard) L-30 (dead publish-one swappool) L-31 (proxy salt derivation) I-16 (key handling, identifier truncation)
docs/DEPLOY.md	H-4 (faucet gating never mentioned) I-16 (key in shell history)

Themes

Four root causes account for most of the list; fixing findings one at a time will leave the others standing:

1. **Contracts trust amounts and answers they were told rather than ones they observed.** C-2 is the sharpest case, and it is why a hostile or non-standard token is dangerous at all. The same pattern recurs wherever a raw .call result is used without validating its length: L-8, M-18, I-5.

2. **Nothing bounds what a dependency may return.** The quoter's multiplier (C-1), the fee policy's rate (H-1, M-5), the protocol controller's rate and recipient (M-5, L-9) and the registry's answer (L-8) are all consumed unvalidated. Several of those contracts have owners distinct from the consumer's, so "the owner would not do that" is not an available argument.
3. **Immutability is claimed but not delivered.** seal on SwapPool (H-2) and on EthFaucet (H-5), the maxStaleness upgrade gap (L-6), and the fees[] claims no balance backs (M-2, M-6, L-2) all publish a guarantee the code does not keep.
4. **Optional security controls default to off, and the docs do not say to turn them on.** tokenRegistry and tokenLimiter on SwapPool (C-2), registry and periodChecker on EthFaucet (H-4), and PeriodSimple's zero period (I-11) all fail open. In the faucet's case the deployment runbook never mentions the wiring step at all, so the insecure configuration is the documented one.

Two cross-cutting fixes would retire a disproportionate share of the list: validating owner != address(0) in all fourteen initializers (L-7), and replacing every raw .call + unchecked-decode with a typed call or a length-validated decode.

Critical severity

C-1 OracleQuoter multiplier above parity drains the pool

Severity	Critical
Target	src/OracleQuoter.sol L25-26, L56-61, L94-95
Proof	test_POC_F01_sameTokenSwap_extractProfit, test_POC_F01_roundTripArbitrage_drainsPool

Description

setMultiplier accepts any value in [900_000, 1_100_000] and valueFor applies it to the **output** side of every quote:

```
uint256 private constant MIN_MULTIPLIER = 900_000; // 0.9x
uint256 private constant MAX_MULTIPLIER = 1_100_000; // 1.1x
...
uint256 effectiveMultiplier = multiplier == 0 ? PPM : multiplier;
return FixedPointMathLib.fullMulDiv(output, effectiveMultiplier, PPM);
```

Because the same factor is applied when quoting $A \rightarrow B$ and when quoting $B \rightarrow A$, it is not a spread — it is a subsidy paid on both legs. Writing $m = \text{multiplier} / \text{PPM}$ and P for oracle prices:

```
valueFor(B, A, v) = v * (P_A / P_B) * m
valueFor(A, B, w) = w * (P_B / P_A) * m
```

so a round trip through a single pool returns $v * m^2$. At $m = 1.1$ that is $1.21 * v$ gross, and the combined pool plus protocol fee would have to exceed 21 % to absorb it. With the documented 1 % pool fee each round trip nets $1.1^2 * 0.99^2 = 1.185921$, so an attacker compounds at 18.6 % per iteration:

Round trip	Attacker balance (from 1 000)
1	1 185.92
2	1 406.41
3	1 667.89
4	1 977.99
5	2 345.73

The loop is repeatable until the pool is empty, and any unprivileged address can run it.

The degenerate case needs no second token at all. Nothing in `_swap` rejects `_inToken == _outToken`, and `valueFor(A, A, v)` reduces to $v * m$, so a single call extracts the subsidy directly:

```
1 000 SRF in -> 1 100 quoted -> 11 pool fee -> 1 089 SRF out. Profit: 89 SRF.
```

The precondition is that the pool owner has set a multiplier above 1_000_000. That is within the contract's permitted range: setMultiplier accepts values up to 1.1x, and docs/SPEC.md presents the multiplier as an ordinary operational knob ("adjust all quotes by a factor in PPM"). Nothing warns that half of the permitted range is unsound.

Recommendation

1. Clamp the multiplier at parity — `MAX_MULTIPLIER = PPM` — so the knob can only ever widen the pool's margin. Values above parity cannot be made safe by fee tuning, because the arbitrage scales with notional while fees are a fixed proportion of it.
2. Independently, reject same-token swaps in `SwapPool._swap`: `if (_inToken == _outToken) revert InvalidToken();`. This closes the single-call variant for every quoter, including third-party ones.
3. If an above-parity subsidy is genuinely wanted, it must be metered per account and per period, not expressed as a

proportion of notional.

C-2 `_deposit` credits the requested amount, not the amount received

Severity	Critical
Target	src/SwapPool.sol L168-174, L286-291
Proof	test_POC_F03_ungatedPool_drainedByWorthlessToken, test_POC_F03_feeOnTransferToken_leaksLiquidity, test_POC_F03_limiterBlocksDrainOnlyIncidentally

Description

`_deposit` checks only the boolean return of `transferFrom`; it never measures how much the pool actually received.

```
bool success = IERC20(_token).transferFrom(msg.sender, address(this), _value);
if (!success) revert TransferFailed();
```

`_swap` then prices the swap on the *requested* `_value`:

```
deposit(_inToken, _value);
uint256 quotedValue = getQuote(_outToken, _inToken, _value);
```

Two impacts follow.

Total drain when neither gate is configured. docs/SPEC.md states that `tokenRegistry` unset means “any token is allowed” and `tokenLimiter` unset means “deposits are uncapped”, and `initialize` accepts `address(0)` for both. In that configuration `mustAllowedToken` and `mustWithinLimit` both return early, so an attacker supplies a token they wrote themselves whose `transferFrom` returns true and moves nothing:

```
function transferFrom(address, address, uint256) external pure returns (bool) {
    return true;
}
```

One call to `withdraw(realToken, fakeToken, poolBalance)` empties the pool. The PoC drains 100 000 tokens for zero cost. A configured `tokenLimiter` incidentally blocks this — an unset limit of 0 fails `balance + delta > limit` — so in that configuration the whitelist is enforced by the limiter, by accident, rather than deliberately.

Continuous leak with fee-on-transfer tokens. For any whitelisted token that takes a cut on transfer, the pool banks less than it pays out on every swap. With a 10 % transfer fee and a 1 % pool fee, 1 000 in leaves the pool holding 900 while it pays out 990 — a 9 % loss of pool value per swap, repeatable.

Recommendation

Measure the balance delta and price the swap on it. This fixes both impacts with one change, because a token that delivers nothing then produces a zero quote:

```
function _deposit(address _token, uint256 _value) private returns (uint256 received) {
    mustAllowedToken(_token, tokenRegistry);
    mustWithinLimit(_token, _value);

    uint256 balanceBefore = IERC20(_token).balanceOf(address(this));
    if (!IERC20(_token).transferFrom(msg.sender, address(this), _value)) revert TransferFailed();
    received = IERC20(_token).balanceOf(address(this)) - balanceBefore;
    if (received == 0) revert TransferFailed();
}
```

`_swap` must then quote on `received`, and `deposit()` must emit `received` in the `Deposit` event rather than `_value`. Separately, consider rejecting `tokenRegistry == address(0)` in `initialize` rather than treating the ungated pool as a supported configuration.

High severity

H-1 Swaps accept no minAmountOut or deadline

Severity	High
Target	src/SwapPool.sol L277-284
Proof	test_POC_F09_ownerFrontRunsSwap_noSlippageProtection

Description

Neither swap entry point takes a minimum-output or expiry argument:

```
function withdraw(address _outToken, address _inToken, uint256 _value) public {
    _swap(_outToken, _inToken, _value, msg.sender);
}
```

The executed price is read from `quoter` and `feePolicy` at execution time, and both are mutable up to the point they are sealed. A caller therefore has no way to bound what they receive. The PoC quotes 990 out for 1 000 in, the owner raises the pair fee to 90 %, and the swap then settles at 100 out — silently, with no revert path available to the user.

Owner-initiated repricing overlaps with powers the owner already has (`withdrawLiquidity` can take the whole pool), so operator trust is unchanged. The High severity comes from the gap being unbounded and unavoidable for everyone else:

- With `OracleQuoter`, the price moves whenever a Chainlink feed publishes a new round. A transaction delayed past a deviation update settles at the new price with no ceiling on the difference.
- `setTokenRegistry` and `setTokenLimiter` are not covered by the seal bitmask at all (see [H-2](#)), so even a fully sealed pool can have a swap turned into a revert mid-flight.
- Any integrator — a wallet, an aggregator, a contract that executes on a router quote — cannot protect its users, because the protection does not exist in the interface. `SwapRouter` computes exact-output amounts that nothing can be done with safely.

Recommendation

Add bounded overloads and keep the existing ones only for backwards compatibility:

```
function withdraw(
    address _outToken,
    address _inToken,
    uint256 _value,
    address _recipient,
    uint256 _minAmountOut,
    uint256 _deadline
) public {
    if (block.timestamp > _deadline) revert Expired();
    uint256 out = _swap(_outToken, _inToken, _value, _recipient);
    if (out < _minAmountOut) revert InsufficientOutput();
}
```

`_swap` should return `netValue` so the check can be made against the amount actually transferred. Emitting `netValue` in the Swap event (currently it emits the gross quotedValue) would also let integrators verify settlement from logs.

H-2 The seal mechanism provides no durable guarantee

Severity	High
Target	src/SwapPool.sol L46, L52, L58, L124-161; cmd/ge-publish/main.go L248
Proof	test_A4_fullySealedPool_registryAndLimiterStillMutable, test_A4b_fullySealedPool_bricked, test_C2b_sealCircumventedByUpgrade_sameKey, test_C1_upgradeAddingSealBit_unsealsExistingPools, test_C2_storagePacking_shiftsSealState (all in AuditPoCSeal.t.sol)

Description

seal exists so an operator can make a public, irreversible commitment that the terms of a pool will not change. docs/SPEC.md describes the bits as permanently locking fields and says they "can only be set, never cleared". That commitment does not hold. It fails four independent ways, and any one of them is sufficient.

1. The two fields that decide which tokens have value are not sealable. The bitmask covers feePolicy, feeAddress and quoter. setTokenRegistry (L155) and setTokenLimiter (L159) have no seal check and no zero-address guard. On a pool where isSealed(0) == true, the owner can still replace both. The PoC seals the pool, confirms the whitelist correctly rejects a junk token, then has the owner set tokenRegistry and tokenLimiter to address(0) — after which **any caller** swaps 100 000 units of a self-minted but otherwise entirely honest ERC20 for the pool's whole real balance at the sealed quoter's 1:1 default.

This is a distinct defect from C-2 and the fix for C-2 does not close it: the token is standards-compliant, the balance delta is exact, and measuring the delta changes nothing. The enabling step is the unsealable setter. As with M-3, the exposure depends on the quoter pricing an unknown token rather than rejecting it — OracleQuoter reverts OracleNotSet and is protected incidentally, while RelativeQuoter, DecimalQuoter and an unset quoter all price at 1:1. The mirror image is also demonstrated: pointing tokenRegistry at a deny-all registry permanently bricks a fully sealed pool.

2. A proxy upgrade defeats the seal, using the same key that set it. sealState is ordinary storage; nothing enforces it after an implementation swap. The PoC seals a pool behind a real ERC1967Factory, confirms setQuoter and setFeeAddress revert Sealed, then performs one factory.upgrade and rewrites quoter, feeAddress and feePolicy while sealState still reads 7.

This failure is decisive rather than theoretical because cmd/ge-publish/main.go defaults the proxy admin to the pool owner:

```
admin := owner
if cfg.Admin != "" { ... }
```

On a default deployment the key that publishes the seal is the key that can remove it, so the seal binds no one.

3. maxSealState is a compiled constant used as the full-seal sentinel. isSealed(0) returns sealState == maxSealState, and maxSealState is constant (L58) rather than storage. The natural fix for point 1 — add a fourth bit and set maxSealState = 15 — silently flips isSealed(0) from true to false on every already-sealed pool and re-opens seal() on pools whose owners believed the process was finished. SealStateChange(_final) changes meaning at the same moment. The PoC upgrades to a V2 differing *only* in maxSealState and shows storage still reading sealState == 7 while isSealed(0) is now false and seal(8) succeeds. **Fix point 1 without accounting for this and you unseal production.**

4. sealState is packed mid-slot with no reserved gap. sealState is the last declared variable and shares slot 10 with feesDecoupled (verified: offset 0 = feesDecoupled, offset 1 = sealState). Inserting a new config flag next to feesDecoupled — the natural place for one, and this repo does append storage in place, as OracleQuoter.multiplier shows — shifts sealState by one byte. The PoC upgrades to a V3 that declares bool paused between the two: sealState() then reads 0, paused() reads true (inheriting the old seal bits), and setQuoter succeeds on what was a sealed pool.

Beyond all four, sealing an address can never freeze behaviour: the sealed feePolicy and quoter are separate contracts with their own owners, who remain free to move the fee to 100 % or move the rates. protocolFeeController and feesDecoupled have no setter at all, and withdrawLiquidity, transferOwnership and renounceOwnership are outside the bitmask entirely.

Recommendation

Seal should either be made real or retired, because its present form invites reliance it cannot support.

1. Bring tokenRegistry and tokenLimiter under the bitmask, and reject address(0) in both setters once sealed.
2. Replace the maxSealState sentinel with an explicit mask so adding a bit is not a silent unseal: `sealState & ALL_BITS == ALL_BITS`, with ALL_BITS stored or versioned, and treat any change to it as a migration.
3. Give sealState its own slot (or an ERC-7201 namespace) and add a `uint256[N] __gap`. Keep everything else append-only.
4. Default `--admin` in `ge-publish` to something other than owner, and document that seal is meaningful only when the proxy admin is a distinct, more conservative principal — a timelock or multisig — than the owner.
5. Document that sealing an address does not freeze that contract's behaviour.

H-3 Splitter's allocation sum wraps in uint32, defeating its only arithmetic invariant

Severity High

Target `src/Splitter.sol` L86-100

Proof `test_POC_splitter_uint32SumOverflow_bypassesAllocationSumCheck`, `test_POC_splitter_overflowedSplit_paysLastRecipientZero`, `test_POC_splitter_overflowedSplit_bricksDistributionAndFreezesFunds` (in `AuditPoCFunds.t.sol`)

Description

`_validateSplit` accumulates PPM allocations into a `uint32`, inside an unchecked block:

```
uint32 sum;
unchecked {
    for (uint256 i; i < accounts.length; ++i) {
        uint32 alloc = percentAllocations[i];
        if (alloc == 0) revert AllocationMustBePositive();
        sum += alloc;
        ...
    }
}
if (uint256(sum) != PERCENTAGE_SCALE) revert InvalidAllocationsSum();
```

The total is computed modulo 2^{32} and only then widened, so the `uint256(sum)` cast is cosmetic — the wrap has already happened. Any allocation set whose true sum is $1_000_000 + k * 2^{32}$ passes the single arithmetic invariant this contract has, and the invalid split is committed to `_splitHash` by `initialize` or `updateSplit`.

It then distributes, because `_distributeETH` never reads `percentAllocations[last]` — the final recipient receives `amountToSplit - running`. So the overshoot can be parked on the last entry and the split still executes, just not as its numbers claim. Two outcomes, both demonstrated:

- **Silent zero payout.** Allocations (500_000, 500_001, 4_294_967_295) have a true sum of $2^{32} + 1_000_000$ and are accepted. Distributing 1 000 wei pays 500, 500, and 0 — the last recipient, nominally holding 4 294 967 295 PPM, gets nothing, and remainder lands on exactly zero so nothing looks wrong.
- **Permanent freeze.** Allocations (900_000, 900_000, 4_294_167_296) share the same true sum. The leading shares now exceed the balance, the second transfer fails, and **both `distributeETH` and `distributeERC20` revert forever**, stranding every asset already in the contract.

`InvalidAllocationsSum` has no test at all, and every allocation in the existing suite is a plausible PPM value between 30 000 and 600 000, so nothing in CI comes near the wrap.

Recommendation

Accumulate in `uint256` and bound each element:

```
uint256 sum;
for (uint256 i; i < accounts.length; ++i) {
    uint32 alloc = percentAllocations[i];
    if (alloc == 0) revert AllocationMustBePositive();
```

```

    if (alloc > PERCENTAGE_SCALE) revert InvalidAllocationsSum();
    sum += alloc;
    ...
}
if (sum != PERCENTAGE_SCALE) revert InvalidAllocationsSum();

```

Removing unchecked alone is not sufficient — it converts the bypass into an unnamed `Panic(0x11)` rather than the documented error. Note the duplicate scan must stay inside its own loop; only the accumulator's type and the unchecked scope need to change.

H-4 EthFaucet fails open, and the documented deployment ships an unlimited public ETH dispenser

Severity	High
Target	src/EthFaucet.sol L75-77, L84-86, L108-125; docs/DEPLOY.md L260-263
Proof	test_EF_defaultConfigHasNoRateLimitSingleAddressDrains, test_EF_settingGatingBackToZeroSilentlyDisablesIt (in AuditPoCToken.t.sol)

Description

Both gates return true when unconfigured:

```

if (periodChecker == address(0)) { return true; }
...
if (registry == address(0)) { return true; }

```

`initialize(owner_, amount_)` sets only the owner and the drip amount, so a freshly deployed faucet has `registry == periodChecker == address(0)` — no whitelist and no cooldown. The design choice is fail-open rather than fail-closed.

That would be a configuration footgun on its own. What makes it a live High is the deployment runbook. `docs/DEPLOY.md:260-263` documents the faucet deployment in full as:

```

./ge-publish deploy-proxy --contract ethfaucet $BASE \
--factory-address $FACTORY --impl-address $IMPL_ETHFAUCET --owner $OWNER \
--faucet-amount 1000000000000000

```

and neither `DEPLOY.md`, `PUBLISH.md` nor `README.md` mentions `setRegistry`, `setPeriodChecker` or `seal` anywhere. `SPEC.md` documents that the setters exist, but nothing tells an operator that calling them is required — so an operator who follows the runbook ends with an ungated faucet, and any CELO subsequently sent to it can be swept by one address in an unbounded loop. The PoC funds a faucet with 10 ether and has a single address take all of it in one transaction.

The owner can also silently re-open a configured faucet by setting either address back to zero; neither setter rejects `address(0)`.

Recommendation

Make the gating fail closed — revert `RegistryBackend` / `PeriodBackend` when the corresponding address is unset, or require both to be non-zero before any claim is served. Reject `address(0)` in `setRegistry` and `setPeriodChecker`. Most importantly, add the wiring step to `docs/DEPLOY.md`: a security control that the deployment guide never mentions is a control that will not be deployed.

H-5 EthFaucet.seal() can permanently lock gating that was never configured

Severity	High
Target	src/EthFaucet.sol L47-53
Proof	test_EF_sealingUnsetGatingPermanentlyDisablesAllRateLimiting (in AuditPoCToken.t.sol)

Description

`seal()` validates only that `_state <= maxSealState` and that no requested bit is already set. It never checks that the field being sealed holds a meaningful value.

Sealing `REGISTRY_STATE` | `PERIODCHECKER_STATE` while both are still `address(0)` — the post-initialize default from [H-4](#) — **permanently and irreversibly disables the whitelist and the cooldown**. `setRegistry` and `setPeriodChecker` then revert Sealed forever. Combined with [M-16](#), which shows there is no withdraw path, the entire balance is permanently free for anyone to take and the owner has no remedy at all.

The seal mechanism exists to give users an immutability guarantee. Here one transaction converts it into a guarantee of the opposite, and the transaction looks like exactly the right thing to do — an operator hardening a fresh deployment before funding it would plausibly seal everything first.

This is the same class of defect as [H-2](#) in `SwapPool1`: sealing an address slot says nothing about whether the slot is safe to freeze.

Recommendation

Require the field to be configured before its bit can be sealed:

```
if (_state & REGISTRY_STATE != 0 && registry == address(0)) revert InvalidState();
if (_state & PERIODCHECKER_STATE != 0 && periodChecker == address(0)) revert InvalidState();
if (_state & VALUE_STATE != 0 && amount == 0) revert InvalidState();
```


Medium severity

M-1 reverseValueFor inverts the rounding stages in the wrong order

Severity Medium

Target src/OracleQuoter.sol L148-173

Proof test_POC_F02_reverseValueFor_breaksRoundtripInvariant, test_POC_F02_reverseValueFor_shortfall_atScale, test_POC_F02_reverseValueFor_violations_areTheNorm, test_POC_F02_router_quoteExactOutput_underDelivers, test_POC_M1_breaksWithIdenticalDecimals, test_POC_M1_correctedOrderRestoresInvariant

Description

IQuoter documents the guarantee `valueFor(out, in, reverseValueFor(out, in, x)) >= x`, and docs/SPEC.md restates it as the SwapRouter “roundtrip property”. It does not hold.

The forward path floors twice — once on the rate conversion, once on the multiplier:

```
uint256 output = determineOutput(...); // floor
uint256 effectiveMultiplier = multiplier == 0 ? PPM : multiplier;
return FixedPointMathLib.fullMulDiv(output, effectiveMultiplier, PPM); // floor
```

The reverse path ceils twice, but applies the two inverses in the *same* order rather than the reverse order:

```
uint256 output = reverseOutput(...); // ceil, rate inverse
return FixedPointMathLib.fullMulDivUp(output, PPM, effectiveMultiplier); // ceil, multiplier inverse
```

To undo `floor(floor(v*A/B)*M/PPM)` you must first undo the *outer* operation. The smallest v that reaches x satisfies `floor(v*A/B) >= ceil(x*PPM/M)`, so the multiplier inverse has to be applied to x **before** the rate inverse. Applying it afterwards adds slack in the wrong place, where the forward path’s first floor has already discarded the precision it was meant to compensate for.

Concrete counter-example, using a 6-decimal token out and an 18-decimal token in at equal prices with `multiplier = 950_000` (well inside the permitted range):

```
reverseValueFor(small, big, 1) = ceil(ceil(1 * 1e12) * 1e6 / 950000)
                               = 1 052 631 578 948
valueFor(small, big, 1 052 631 578 948)
      = floor(floor(1052631578948 / 1e12) * 950000 / 1e6)
      = floor(1 * 0.95) = 0      -> 0 < 1
```

This is not an isolated edge case: 95 of the first 100 request sizes under-deliver in that configuration, and the shortfall persists at scale — asking for 100e6 yields 99 999 999. Because `SwapPool.getAmountIn`’s +1 safety margin is denominated in *input* units, and one wei of an 18-decimal input is 1e-12 of a 6-decimal output, it cannot cover the gap. The PoC shows the failure reaching `SwapRouter.quoteExactOutput` for a zero-fee pool, where `_reverseNetToQuoted` contributes no ceiling slack of its own to mask it.

The trigger is not the decimal gap specifically. Writing $A = \text{inRate} * \text{outScale} * \text{outRateScale}$ and $B = \text{inRateScale} * \text{inScale} * \text{outRate}$, the defect appears whenever B / A is large enough that the forward `floor(v * A / B)` truncates coarsely. A decimal mismatch is only one way to get there — an extreme *price* ratio does it too. With both tokens at 6 decimals, both feeds at 8 decimals, and the input token priced at 1e-6 of the output token, $B / A = 1e6$ and `reverseValueFor(out, in, 1) = 909_091` still forwards to 0. Any pool pairing assets of very different unit value is exposed regardless of decimals, so the fix cannot be scoped to cross-decimal pairs.

With the multiplier unset or at exactly PPM, `determineOutput` and `reverseOutput` are a correct adjoint pair and the invariant holds for every amount tested. The rate math itself is sound; the defect is purely the stage ordering.

The absolute shortfall is small — one unit of the output token’s smallest denomination — so this is a broken invariant rather than a drain. It matters because exact-output routing is built on the guarantee: an integrator that funds a swap with `quoteExactOutput` will come up short, and any downstream `require(received >= requested)` reverts.

Recommendation

Undo the multiplier first, then the rate:

```
uint256 effectiveMultiplier = multiplier == 0 ? PPM : multiplier;
uint256 target = FixedPointMathLib.fullMulDivUp(_value, PPM, effectiveMultiplier);
return reverseOutput(target, din, dout, inRate, inRateDecimals, outRate, outRateDecimals);
```

With the counter-example above: $\text{target} = \text{ceil}(1\text{e}6 / 950000) = 2$, so $\text{reverseOutput}(2) = 2\text{e}12$, and $\text{valueFor}(2\text{e}12) = \text{floor}(2 * 0.95) = 1 \geq 1$. `test_POC_M1_correctedOrderRestoresInvariant` applies this corrected ordering to all 22 amounts that break under the shipped order and confirms every one of them satisfies the invariant.

Add round-trip fuzz coverage in both decimal directions *and* across a wide price ratio — the existing fuzz tests only exercise a pair where the intermediate division happens to be exact (see [Test coverage gaps](#)).

M-2 fees[] credited after the outbound transfers permits reentrant fee inflation

Severity	Medium
Target	src/SwapPool.sol L286-318
Proof	test_POC_F04_reentrancy_corruptsDecoupledFeeAccounting

Description

`_swap` violates checks-effects-interactions: it transfers the protocol fee and the user's output out of the pool, and only then records the pool fee.

```
if (protocolFee > 0) {
    if (!IERC20(_outToken).transfer(_getProtocolRecipient(), protocolFee)) revert TransferFailed();
}
if (!IERC20(_outToken).transfer(_recipient, netValue)) revert TransferFailed();

if (totalFee > 0 && feeAddress != address(0)) {
    fees[_outToken] += totalFee; // effect, after two interactions
}
```

There is no `nonReentrant` guard anywhere in the contract. When `feesDecoupled == true`, available liquidity is computed as `balance - fees[token]`, so during the reentrancy window the outer swap's fee still looks like free liquidity. A `tokenOut` with a transfer hook (ERC-777, ERC-1363, or any custom token that notifies on transfer) re-enters and spends it.

The PoC uses a 1 000-token pool with a 10 % fee. The outer swap pays out 900 and leaves 100 behind as its fee; the reentrant call sees those 100 as available, swaps them out, and credits its own 10. When the outer frame resumes it adds its 100:

```
fees[tokenOut] = 110
balanceOf(pool, tokenOut) = 10
```

The attacker does not profit — they paid 1 100 in and received 990 out — so this is griefing, not theft. The damage is to the pool:

- `withdraw(tokenOut)` (collect all fees) reverts forever, so the owner's recorded 110 is unrecoverable through the fee path.
- Available liquidity is `bal > fees ? bal - fees : 0`, which is now 0, so **every future swap of that token reverts with InsufficientBalance**. The token is permanently disabled; the only recovery is `withdrawLiquidity` plus a redeployment.

Recommendation

Move the effect before the interactions and add a guard:

```
if (totalFee > 0 && feeAddress != address(0)) {
    fees[_outToken] += totalFee;
}
if (protocolFee > 0) { ... transfer ... }
if (!IERC20(_outToken).transfer(_recipient, netValue)) revert TransferFailed();
```

Add `solady/Utils/ReentrancyGuard` and mark `deposit`, `_swap`'s public entry points, and the two fee-collection overloads `nonReentrant`. Reordering alone is sufficient for this specific path, but the guard is what makes the contract safe against arbitrary tokens in general, which is the position it is in whenever `tokenRegistry` admits community-issued tokens.

M-3 `tokenOut` is never checked against `tokenRegistry`

Severity	Medium
Target	<code>src/SwapPool.sol</code> L286-291, L377-386
Proof	<code>test_POC_F05_tokenOut_bypassesRegistryWhitelist</code>

Description

`_swap` validates only the input leg. `mustAllowedToken` is reached through `deposit(_inToken, _value)`; `_outToken` flows straight into `getQuote`, the balance check and transfer with no registry lookup. The whitelist is therefore one-directional.

Any token the pool happens to hold can be swapped out at whatever terms the quoter produces, whether or not the operator ever authorised it for trading. That covers tokens sent to the pool by mistake or as a donation, and tokens deliberately removed from the registry — de-listing stops deposits but does not stop extraction. The PoC confirms that depositing the unregistered token reverts with `UnauthorizedToken` while receiving it out of the pool succeeds.

Severity is capped by the quoter: `OracleQuoter` reverts with `OracleNotSet` for a token it has no feed for, so pools using it are protected incidentally. Pools using `RelativeQuoter` (unset price index defaults to PPM), `DecimalQuoter`, or no quoter at all price unknown tokens at 1:1 and are exposed.

Recommendation

Validate both legs in `_swap`:

```
mustAllowedToken(_outToken, tokenRegistry);
```

If allowing exits from a de-listed token is deliberate, make it explicit with a separate de-listing state rather than leaving it as a side effect of asymmetric validation.

M-4 A single global `maxStaleness` cannot serve feeds with different heartbeats

Severity	Medium
Target	<code>src/OracleQuoter.sol</code> L30, L51-54, L106
Proof	<code>test_POC_M4_globalStaleness_cannotFitHeterogeneousFeeds</code>

Description

`maxStaleness` is one value applied to every feed:

```
if (block.timestamp - updatedAt > maxStaleness) revert StaleOraclePrice(oracle);
```

But the whole design of the quoter is to cross-rate arbitrary tokens through a shared denomination, and `docs/SPEC.md`'s own setup guide mixes feeds with very different update characteristics in a single quoter — KES/USD, USDT/USD, CUSD/USD and BRL/USD on Celo. Chainlink heartbeats differ per feed, from minutes for major pairs to a day for thin FX pairs.

There is therefore no value the owner can choose that is simultaneously correct for all of them, and the PoC demonstrates both horns:

- At the 86400 default, a feed with a one-hour heartbeat that has been **dead for 23 hours** — 23 consecutive missed updates, so the price is arbitrarily wrong — is still accepted and priced into quotes.
- Tightening to 3600 to protect that feed makes a perfectly healthy 24-hour-heartbeat feed, updated two hours ago, revert `StaleOraclePrice` and take every pool that depends on it offline.

Either the operator tolerates arbitrarily stale prices on fast feeds, or they break slow feeds. Combined with the absence of slippage protection (H-1), the first choice is a standing mispricing vector against pool liquidity rather than a theoretical one.

Recommendation

Store staleness per feed, set alongside the oracle so the two cannot drift apart, with the global value as a fallback:

```
mapping(address => uint256) public oracleMaxStaleness;

function setOracle(address token, address oracleAddress, uint256 maxStaleness_) public onlyOwner { ... }
```

M-5 The fee domain is a joint constraint nobody enforces, and at its boundary a swap pays out zero

Severity	Medium
Target	src/SwapPool.sol L195-200, L264-275, L286-318
Proof	test_POC_F06_highPoolFee_bricksSwapsAndQuotes; test_F5_denominatorZero_forwardSucceedsReversePanics, test_F6_protocolFeeOwner_bricksReverseQuotesOfLivePools, test_F7_boundaryTable (in AuditPoCSeal.t.sol)

Description

The protocol fee is charged *on top of* the pool fee, and neither direction tolerates the sum reaching the quote:

```
uint256 netValue = quotedValue - totalFee - protocolFee; // underflows
...
denominator = ppmSquared - _feePpm * (PPM + _protocolFeePpm); // underflows, or is zero
```

Both paths are therefore only defined for $\text{feePpm} * (\text{PPM} + \text{protocolFeePpm}) < \text{PPM}^2$. Nobody enforces that: FeePolicy caps fee at PPM and ProtocolFeeController caps protocolFee at PPM, independently, and the two are **separate contracts with separate owners**. Neither can see the other's value. The measured domain:

protocolFeePpm	maximum usable feePpm
0	999 999
100 000 (10 %)	909 089
250 000 (25 %)	799 999
700 000 (70 %)	588 234
1 000 000 (100 %)	499 999

Three consequences, in increasing order of severity.

A one-PPM cliff with an unattributable error. At a 10 % protocol fee, $\text{feePpm} = 909\,090$ works and $909\,091$ makes `getAmountOut`, `getAmountIn` and `withdraw` all revert with a bare `Panic(0x11)`. The operator gets no signal that the value is out of range.

At exact equality the two directions disagree, and the user is charged for nothing. With $\text{feePpm} = 800\,000$ and $\text{protocolFeePpm} = 250\,000$ the product is exactly PPM^2 , so the denominator is zero rather than negative. `getAmountIn` reverts `Panic(0x12)` (division by zero — a different panic from the underflow above), but `getAmountOut` returns **0 with no revert**, and `withdraw(B, A, 100e18)` **succeeds**: the caller's entire input is taken and zero is transferred back. That is direct, silent loss of user funds on a successful transaction, not a denial of service. It shares a root cause with L-5 — nothing floors the output at a non-zero value.

The protocol can retroactively brick live pools. A pool sitting at a 60 % fee quotes correctly while the protocol fee is 10 %. The `ProtocolFeeController` owner — a different principal from the pool owner, with no visibility into any pool's fee — calls `setProtocolFee(700_000)`, and every `getAmountIn`, `getAmountOut` and `withdraw` on that pool now panics. One party can disable another's pools by changing a value that is legal in its own contract.

The $\text{feePpm} < \text{DEFAULT_FEE_PPM}$ branch is never affected: its maximum deduction is $2e10$ against $1e12$.

Recommendation

Validate the joint constraint inside `SwapPool`, where both values are visible, and fail with a named error rather than a

panic:

```
error FeeTooHigh();
...
if (_feePpm * (PPM + _protocolFeePpm) >= PPM * PPM) revert FeeTooHigh();
```

Apply it in `_reverseNetToQuoted` and again in `_swap` before computing `netValue`, and add `if (netValue == 0) revert InsufficientOutput();` so the pay-nothing case cannot succeed. Do not rely on `FeePolicy` or `ProtocolFeeController` to enforce a bound that neither can evaluate alone.

M-6 `withdrawLiquidity` ignores `fees[]`, taking reserved fees and bricking the token

Severity	Medium
Target	src/SwapPool.sol L367-375
Proof	test_B1_withdrawLiquidity_stealsReservedFees_andDoSesPool, test_B2_feeAddressSeal_doesNotProtectFees (in AuditPoCSeal.t.sol)

Description

`withdrawLiquidity` bounds the amount by the raw balance and never consults `fees[]`:

```
uint256 balance = IERC20(token).balanceOf(address(this));
if (amount > balance) revert InsufficientBalance();
```

In `feesDecoupled` mode `fees[token]` is an explicit reservation for `feeAddress`, which is a different principal from the owner — that separation is the entire reason `FEEADDRESS_STATE` exists. Sweeping the full balance takes it, and because nothing adjusts `fees[]`, the pool is left with `fees[token] > balanceOf(token)` **permanently**.

The PoC accrues 10e18 of reserved fees from one swap, has the owner sweep the whole 910e18 balance, and then shows both consequences: `withdraw(B)` (collect all) reverts forever, and since decoupled available liquidity is `bal > fees ? bal - fees : 0`, every future swap into that token reverts `InsufficientBalance` — **even after a fresh 9e18 donation**. The token is permanently disabled, not merely emptied. A second test confirms this all works with `seal(7)` applied, so the `feeAddress` seal protects nothing it claims to.

`docs/SPEC.md` documents `withdrawLiquidity` as an owner emergency lever, so the owner taking the balance is intended. What is not intended is leaving an unbacked claim in `fees[]` that bricks the token for everyone afterwards. This is the third independent route to `fees[] > balance`, alongside the reentrancy in M-2 and the coupled-mode re-lending in L-2, and unlike those it needs no hostile token and no reentrancy.

Recommendation

Either exclude reserved fees from what the owner can sweep, or make the clawback explicit in the accounting:

```
uint256 available = balance > fees[token] ? balance - fees[token] : 0;
if (amount > available) revert InsufficientBalance();
```

If sweeping reserved fees must remain possible for a genuine emergency, decrement `fees[token]` by the shortfall in the same call so the invariant `fees[token] <= balanceOf(token)` is never broken.

M-7 Permissionless deposit plus a total-balance cap is a cheap denial of service

Severity	Medium
Target	src/SwapPool.sol L163-166, L388-396
Proof	test_E1_permissionlessDeposit_capSaturationDoS (in AuditPoCSeal.t.sol)

Description

`deposit` has no access control, and `mustWithinLimit` caps the pool's **total standing balance** rather than the incoming delta:

```
uint256 limit = ILimiter(tokenLimiter).limitOf(_token, address(this));
```

```
uint256 balance = IERC20(_token).balanceOf(address(this));
if (balance + _valueDelta > limit) revert LimitExceeded();
```

Anyone can therefore top a token up to its cap, after which every swap using that token as *input* reverts `LimitExceeded` — for any size, including 1 wei, indefinitely. The PoC sets a 1 000e18 cap on a pool holding 900e18, has the attacker deposit 100e18, and then shows honest swaps of both 1 wei and 1e18 reverting.

The attack is close to free. The attacker’s deposit is not burned: it becomes pool liquidity, and they can swap it back out in the other direction whenever they like. The PoC does exactly that at the end. Total cost is roughly one round-trip fee, and it can be repeated.

The limiter is optional, so this affects only pools that configure one — but those are precisely the pools that took the trouble to bound their exposure. Note also that a limiter is currently the only thing standing between an ungated pool and C-2, so “remove the limiter” is not an available mitigation.

Recommendation

Cap the incoming delta of a deposit rather than the resulting standing balance, so a saturated balance blocks new liquidity without blocking swaps. If the cap genuinely needs to bound total exposure, treat “at cap” as “no new deposits” while still permitting swaps that reduce the capped balance, and consider gating bare deposit to an allowlist so only swaps can move a token toward its cap.

M-8 RelativeQuoter divides before multiplying, breaking the same round-trip guarantee as M-1

Severity	Medium
Target	src/RelativeQuoter.sol L32-67, L69-75, L78-113
Proof	test_POC_relativeQuoter_breaksRoundtripInvariant, test_POC_relativeQuoter_roundtripViolations_areTheNorm, test_POC_relativeQuoter_precisionLoss_isAWholeOutputUnit (in AuditPoCQuoter.t.sol)

Description

`valueFor` normalises decimals **before** applying the exchange rate, so when the input token has more decimals than the output it truncates the input to the output token’s granularity first and only then multiplies:

```
uint256 d = din > dout ? 10 ** ((din - dout)) : 10 ** ((dout - din));
if (din > dout) {
    return determineOutput(_value / d, inExchangeRate, outExchangeRate);
}
```

That is division before multiplication, and it costs real value rather than a rounding wei. The discarded remainder is (`_value mod d`), and the rate is applied afterwards, so the loss in output units is bounded by `inRate / outRate` — a whole output token whenever the input token is rated a million times the reference. The PoC configures an 18-decimal token at 1e6 x against a 6-decimal token at 1.0, quotes 2e12 - 1 wei, and gets 1.000000 where the correct answer is 1.999999: **999 999 units lost on a single quote**.

`reverseValueFor` cannot undo it. It multiplies by `d` before inverting the rate, which is the right order, but the forward path’s leading `floor(_value / d)` has already destroyed information no ceiling can restore. With the input token rated 3.0 and the output at 1.0:

```
reverseValueFor(out6, in18, 1) = ceil(1 * 1e12 / 3) = 333 333 333 334
valueFor(out6, in18, 333 333 333 334) = floor(333 333 333 334 / 1e12) * 3 = 0
```

Zero delivered against one requested. 20 of the first 30 request sizes fail the same way. This is the identical broken invariant as M-1 — the documented `IQuoter` guarantee and the `SwapRouter` round-trip property — reached through a different root cause, so **fixing M-1 in OracleQuoter leaves this standing**. Any pool configured with `RelativeQuoter` has the same defective exact-output routing.

`DecimalQuoter` was checked for the same defect and does **not** have it: its reverse is a correct inverse in both decimal directions (`test_decimalQuoter_roundtrip_holds`). It still truncates dust to zero on the forward path, which is L-5.

Recommendation

Fold the decimal adjustment and the rate into a single `mulDiv` so no intermediate value is ever truncated:


```
// din > dout
return FixedPointMathLib.fullMulDiv(_value, inExchangeRate, outExchangeRate * d);
// dout > din
return FixedPointMathLib.fullMulDiv(_value * d, inExchangeRate, outExchangeRate);
```

and mirror it with `fullMulDivUp` in `reverseValueFor`. `OracleQuoter` already uses `FixedPointMathLib` for exactly this reason; `RelativeQuoter` should too. Add the same cross-decimal, wide-rate-ratio round-trip fuzz recommended in M-1.

M-9 `AccountsIndex.deactivate()` does not revoke the whitelist it appears to control

Severity	Medium
Target	<code>src/AccountsIndex.sol</code> L104-112, L123-129; consumed at <code>src/SwapPool.sol</code> L382 and <code>src/EthFaucet.sol</code> L89
Proof	<code>test_AI_deactivate_doesNotRevokeTheHaveWhitelist</code> (in <code>AuditPoCIndex.t.sol</code>)

Description

`AccountsIndex` exposes two membership predicates that disagree about deactivation:

```
function have(address _account) external view returns (bool) {
    return entryIndex[_account] > 0; // ignores BLOCKED_FIELD
}

function isActive(address _account) external view returns (bool) {
    return this.have(_account) && (entryIndex[_account] & BLOCKED_FIELD) != BLOCKED_FIELD;
}
```

Both consumers in the codebase call `have(address)` and neither calls `isActive`. `SwapPool.mustAllowedToken` issues `abi.encodeWithSignature("have(address)", _token)` and `EthFaucet` does the same for its registry whitelist.

So `deactivate()` is a no-op against the only thing the registry is used for. An operator suspending a compromised, sanctioned or abusive entry sees every signal of success — `isActive()` flips to false, `AddressActive(account, false)` is emitted, the dashboard shows it disabled — while that entry continues to pass the faucet whitelist and the `SwapPool` token gate. The PoC makes the exact raw `have(address)` call the consumers make and shows it still decoding to true after `deactivate()`.

The only effective revocation is `remove()`, which is a different operation with different semantics (it frees the slot and loses the timestamp). Strictly this is an interface-contract mismatch rather than a bug inside `AccountsIndex`, but the resulting failure mode is a whitelist that does not deny.

Recommendation

Make the authorization primitive activation-aware, which is the safer default because `have()` is what enforcement reads:

```
function have(address _account) external view returns (bool) {
    return this.isActive(_account);
}
```

If `have()` must keep meaning “present”, change both consumers to call `isActive`. Either way, state in `docs/SPEC.md` which of the two is the authorization primitive — it currently documents both without saying which one gates access.

M-10 An empty token symbol creates a permanent, unremovable ghost entry

Severity	Medium
Target	<code>src/TokenUniqueSymbolIndex.sol</code> L68-79, L100, L152-154
Proof	<code>test_TI_emptySymbolToken_createsUnremovableGhostEntry</code> (in <code>AuditPoCIndex.t.sol</code>)

Description

bytes32(0) is simultaneously a legal symbol key for _register and the "absent" marker for membership:

```
function have(address _token) external view returns (bool) {
    return tokenIndex[_token] != bytes32(0x0);
}
```

A token whose symbol() returns "" produces symbolKey == bytes32(0). The duplicate check if (idx != 0) revert SymbolAlreadyExists() passes, because registry[bytes32(0)] is 0, so _register pushes the token into tokens and identifierList, sets registry[bytes32(0)] = tokens.length, and then writes tokenIndex[_token] = bytes32(0) — which is indistinguishable from never having been registered.

The result is an entry that is counted by entryCount(), returned by entry() and resolvable through addressOf(bytes32(0)), but denied by have() and **unremovable by anyone including the owner**, because remove() gates on tokenIndex[_token] == bytes32(0) and reverts NotFound. The bytes32(0) symbol slot is squatted permanently, and enumeration — which off-chain indexers use to render the token list — diverges forever from the whitelist SwapPool enforces.

The direction fails closed: a ghost is not whitelisted, so this corrupts the registry rather than opening it. That is why it is Medium and not High. Recovery requires a proxy upgrade.

Recommendation

Reject the sentinel where the key is formed:

```
function _register(address _token, bytes32 _symbolKey) internal {
    if (_symbolKey == bytes32(0)) revert EmptySymbol();
    ...
}
```

M-11 TokenUniqueSymbolIndex de-duplicates symbols but not tokens, permanently squatting symbol keys

Severity Medium

Target src/TokenUniqueSymbolIndex.sol L68-79, L40-42, L100-112

Proof test_TI_symbolChangingToken_doubleRegisters_
thenCorruptsHaveVersusEnumeration, test_TI_initializeWithDuplicateToken_
corruptsHaveAfterRemove (in AuditPoCIndex.t.sol)

Description

tokenIndex is a one-to-one token→symbol map, but _register only checks for a duplicate **symbol**:

```
uint256 idx = registry[_symbolKey];
if (idx != 0) revert SymbolAlreadyExists(); // checks the symbol...
...
tokenIndex[_token] = _symbolKey; // ...and overwrites the token's entry
```

Nothing stops one address from occupying several slots. A token whose symbol() changes — an upgradeable token, or a rebrand, both ordinary reasons for a writer to re-register it — gets added twice: tokens = [0, T, T], registry[AAA] = 1, registry[BBB] = 2, and tokenIndex[T] remembers only BBB.

remove(T) then resolves i = registry[tokenIndex[T]] = 2, deregisters that one slot, and clears tokenIndex[T]. Afterwards have(T) is false while entry(0) still returns T, identifier(0) still returns AAA, and addressOf(AAA) still resolves to T. The stale slot can never be removed, and because registry[AAA] is never freed, **the AAA symbol key is squatted permanently — no token, not even T itself, can ever be registered under it again.**

That last consequence is why this is Medium rather than a bookkeeping note. A user who can get two ordinary registration requests actioned can burn a valuable ticker (USDC, KES, cUSD) in the index forever. The same corruption is also reachable with no unusual token at all, by passing a repeated address to initialize.

Recommendation

Make registration reject an already-indexed token, and validate initialize:


```
if (tokenIndex[_token] != bytes32(0)) revert TokenAlreadyExists();
```

In initialize, require `initialTokens.length == initialSymbols.length` (see L-14) and that no address or symbol repeats. Re-registering a renamed token should be an explicit remove then register, not an implicit second slot.

M-12 Splitter accepts address(0) and itself as recipients

Severity	Medium
Target	src/Splitter.sol L82-101
Proof	test_POC_splitter_acceptsZeroAddressRecipient_burnsETH, test_POC_splitter_zeroAddressRecipient_bricksERC20Distribution, test_POC_splitter_acceptsItselfAsRecipient_neverFullyDistributes (in AuditPoCFunds.t.sol)

Description

`_validateSplit` checks length, duplicates, positive allocations and the sum, but never checks that a recipient address is usable — unlike `RescueVault._checkTo`, which does exactly this for its single destination.

`SafeTransferLib.safeTransferETH(address(0), share)` succeeds, because the zero address is an ordinary account, so a split containing `address(0)` **irreversibly burns that recipient's percentage of every ETH deposit** while `distributeETH` returns success and the balance goes to zero. For ERC20 the failure inverts: any standards-compliant token that rejects transfers to `address(0)` makes `distributeERC20` revert permanently, so that token's balance can never be distributed at all.

The same gap admits `address(this)`. The splitter becomes its own recipient, pays its share to itself, and silently breaks the documented "distribute the entire balance" contract — the PoC shows 0.6 ether still sitting in the contract after eleven distributions.

Recommendation

```
if (accounts[i] == address(0) || accounts[i] == address(this)) revert ZeroAddress();
```

M-13 One hostile or broken recipient blocks Splitter distribution for everyone

Severity	Medium
Target	src/Splitter.sol L112, L116, L131, L135
Proof	test_POC_splitter_oneRevertingRecipient_blocksEveryoneElse, test_POC_splitter_gasBurningRecipient_blocksEveryoneElse, test_POC_splitter_zeroOwnerPlusHostileRecipient_freezesFundsForever (in AuditPoCFunds.t.sol)

Description

Distribution is one atomic loop of push payments with no per-recipient failure isolation, no pull fallback, and no try wrapper. `SafeTransferLib.safeTransferETH` forwards **all remaining gas** and reverts on failure, so a recipient that reverts in `receive()` makes `distributeETH` unconditionally revert for the entire recipient set, holding the whole balance hostage.

The gas-burning variant is worse: because gas is forwarded rather than stipended, a recipient with an infinite loop in `receive()` cannot be routed around by raising or lowering the caller's gas limit. The PoC fails at a 3 000 000 gas budget.

The only escape is `updateSplit`, which is `onlyOwner` — so combined with L-7, a splitter initialised with `owner == address(0)` and one hostile recipient is frozen permanently with no reachable remedy. That is the escalation that makes L-7 worth fixing rather than merely noting.

Recommendation

Use `SafeTransferLib.forceSafeTransferETH(accounts[i], share, gasStipend)`, which falls back to a force-send and cannot be grieved, or move to a claim model where a failed transfer is credited to a per-account balance the recipient withdraws itself. For ERC20, the pull model is the only robust option.

M-14 Permissionless distribution plus "remainder to the last recipient" lets that recipient capture a dripped stream

Severity	Medium
Target	src/Splitter.sol L44, L110-116
Proof	test_POC_splitter_permissionlessDripDistribution_stealsEntireStream (in AuditPoCFunds.t.sol)

Description

Three individually reasonable decisions combine badly: `_scaleAmountByPercentage` floors; a non-final share of zero is skipped by `if (share != 0)`; and everything the leading recipients round away becomes remainder, which goes to `accounts[last]`.

Because `distributeETH` is permissionless and always operates on the *entire current balance*, the granularity of distribution is chosen by whoever calls it — including the last recipient. For any balance b where $b * p_i / 1_000_000 < 1$, recipient i is paid nothing and the last recipient takes everything.

The PoC uses a (999_999 PPM, 1 PPM) split and drips 200 deposits of 1 wei, with the 1-PPM recipient calling `distributeETH` after each one: **the 999_999-PPM recipient receives 0 and the 1-PPM recipient receives all 200 wei**. The control in the same test distributes the identical 200 wei in one call and correctly pays

1. No privilege, no reentrancy — only gas.

Recommendation

Apportion the remainder proportionally (largest-remainder / Hamilton) rather than assigning it to a fixed index, and require `grossAmount >= PERCENTAGE_SCALE` before distributing so no per-recipient share can round to zero. Restricting the distribute functions to a keeper removes the caller's control over granularity but leaves the underlying bias in place, so it is not a substitute.

M-15 Splitter.updateSplit retroactively redirects funds already deposited

Severity	Medium
Target	src/Splitter.sol L39-42
Proof	test_POC_splitter_ownerRetroactivelyRedirectsDepositedFunds (in AuditPoCFunds.t.sol)

Description

`updateSplit` replaces the commitment immediately — no timelock, no delay, and no requirement that the pending balance be distributed under the old split first. Since distribution always targets the live balance, the owner can wait for value to accumulate under the split recipients agreed to and then re-point it in one transaction. The validation rules are trivially satisfied by (owner: 999_999, sink: 1).

The PoC accumulates 10 ether under a (60 %, 40 %) split and has the owner take 9.99999 ether. Any in-flight `distributeETH` carrying the old arrays now reverts `InvalidHash`, so the honest distribution can also be front-run out of existence.

A splitter's entire purpose is to be a credible commitment to its recipients, and `getHash` exists so they can verify it. That verification is worth nothing if the commitment can be replaced over a balance that already exists.

Recommendation

Require the balance to be zero before `updateSplit` takes effect, or auto-distribute under the old split first, or gate the

change behind a commit/apply timelock so recipients have a window to distribute.

M-16 EthFaucet has no withdraw path, so ETH can be locked permanently

Severity	Medium
Target	src/EthFaucet.sol L45, L55-60
Proof	test_EF_noWithdrawPathSealedAmountLocksEthForever, test_EF_dustBelowAmountIsUnreachable (in AuditPoCToken.t.sol)

Description

The faucet accepts ETH through `receive()` but exposes no owner `withdraw`, `sweep` or `rescue` function. The only egress is `gimme` / `giveTo`, both of which require `amount <= address(this).balance`. Two consequences:

- Any residual balance strictly below `amount` is unreachable forever.
- If the owner seals `VALUE_STATE` while `amount` exceeds the balance the faucet will ever hold, `setAmount` reverts Sealed permanently and **100 % of the balance is locked with no recovery path**.

The PoC leaves 0.5 ether in a faucet with `amount == 1 ether`, seals `VALUE_STATE`, and then probes seven plausible `withdraw` selectors — `withdraw()`, `withdraw(uint256)`, `withdrawAll()`, `sweep()`, `rescue()`, `drain()`, `emergencyWithdraw()` — all of which fail. `RescueVault` cannot help either, since extracting the ETH would still require the faucet to send it.

Recommendation

Add function `withdraw(address payable to, uint256 value)` external `onlyOwner`, or at minimum a `sweep()` to the owner. Paying out `min(amount, balance)` would also make dust drainable through normal claims.

M-17 giveTo rate-limits the recipient, not the caller, so recipient rotation defeats the cooldown

Severity	Medium
Target	src/EthFaucet.sol L134-139
Proof	test_EF_giveToRecipientRotationDrainsFaucetDespiteCooldown (in AuditPoCToken.t.sol)

Description

`giveTo` is permissionless and the cooldown is keyed on the *recipient*. There is no per-caller limit and no global budget, so one actor holding `N` fresh addresses empties the faucet in a single transaction no matter how long the period is. `PeriodSimple.balanceThreshold` offers nothing here, because fresh addresses start at zero balance — the threshold is designed to skip *funded* accounts.

The PoC configures a 1-day period and a 0.5-ether balance threshold, then drains 10 ether across 10 fresh recipients in one transaction. A repeat `giveTo` to the first recipient correctly reverts `PeriodBackend`, confirming the cooldown works exactly as specified — it simply protects nothing in aggregate.

This is inherent to a faucet with no whitelist, which is why it compounds [H-4](#): the registry is the only thing that makes the cooldown meaningful, and the registry is optional and undocumented in the deploy path.

Recommendation

Treat the registry as mandatory rather than optional, and add a global per-window dispensing cap (`maxPerPeriod`) so the faucet's total outflow is bounded independently of how many identities a claimant controls.

M-18 The faucet's registry and period calls never validate returndata

Severity	Medium
Target	src/EthFaucet.sol L79-81, L89-91, L121-123
Proof	test_EF_registryWithNoCodePanicsInsteadOfRegistryBackend, test_EF_periodCheckerWithNoCodePanicsInsteadOfPeriodBackend, test_EF_registryReturningShortDataPanics, test_EF_registryReturnValueOnlyInspectsByte31 (in AuditPoCToken.t.sol)

Description

All three gating calls are raw `.calls` whose result is then indexed without any length check:

```
(bool ok, bytes memory result) = registry.call(abi.encodeWithSignature("have(address)", _recipient));
if (!ok) revert RegistryBackend();
return result[31] == 0x01;
```

Two distinct defects.

Codeless or short-returning targets panic instead of reverting with the dedicated error. A call to an address with no code *succeeds* with empty returndata, so `result[31]` reverts `Panic(0x32)` — array out of bounds — rather than `RegistryBackend.check()`, which is specified to return a `bool`, reverts too. This is the same failure shape as L-8 and I-5: a named error written for exactly this misconfiguration cannot fire for it.

Only one byte of truthiness is inspected. `result[31] == 0x01` means a word that `abi.decode(_, (bool))` would reject — `257 == 0x...0101` — is accepted as `true` and grants a claim, while a genuinely truthful word like `256 == 0x...0100` reads as `false`. `_checkAndPoke` at L123 uses `!= 0` on the same byte, a third convention across three call sites in one contract.

Recommendation

Use typed interface calls, or validate and decode properly:

```
if (!ok || result.length < 32) revert RegistryBackend();
return abi.decode(result, (bool));
```

Add an `extcodesize` check in `setRegistry` and `setPeriodChecker` so a codeless address is rejected at configuration time.

M-19 GiftableToken's expired flag and Expired event never persist

Severity	Medium
Target	src/GiftableToken.sol L86-99, L110-112
Proof	test_GT_expiredFlagNeverPersistsViaTransferPath, test_GT_expiryIsNeverRecordedByUsersHittingTheRevert (in AuditPoCToken.t.sol)

Description

The transfer hook calls `applyExpiry()` and reverts on any non-zero return:

```
function _beforeTokenTransfer(address, address, uint256) internal virtual override {
    if (applyExpiry() != 0) revert TokenExpired();
}
```

But `applyExpiry()` returns 2 from precisely the branch that performs the state change:

```
if (block.timestamp >= _expires) {
    expired = true;
    emit Expired(block.timestamp);
    return 2;
}
```

So the SSTORE and the log are rolled back with the rest of the reverted call. Every write path that could flip the flag — transfer, transferFrom, mintTo, burn — reverts, which means expired can **only** ever be set by a standalone applyExpiry() call.

docs/SPEC.md:50 states that "Expiry flips automatically on the first transfer at or after block.timestamp >= expiresAt". It does not. Off-chain indexers reading expired() or watching for Expired see a live token indefinitely, and whoever eventually calls applyExpiry() stamps the event with *their* timestamp rather than the real expiry moment — so the event is not a reliable record of when expiry occurred.

Transfer blocking itself is unaffected, because it is recomputed from block.timestamp on every call. That makes expired a pure cache — and one whose only effect is to put a potentially state-writing, non-view call on the hot transfer path.

Recommendation

Make the hook a pure read and let applyExpiry() be the sole writer:

```
function _beforeTokenTransfer(address, address, uint256) internal virtual override {
    if (_expires != 0 && (expired || block.timestamp >= _expires)) revert TokenExpired();
}
```

If lazy out-of-band updating is acceptable, correct SPEC.md:50 instead — but the two must agree.

M-20 burn() is blocked after expiry, so an expired token's supply can never be retired

Severity	Medium
Target	src/GiftableToken.sol L101-108
Proof	test_GT_burnRevertsAfterExpiry_supplyCanNeverBeRetired (in AuditPoCToken.t.sol)

Description

burn calls solady's _burn, which invokes _beforeTokenTransfer(from, address(0), amount) — which reverts once expired. The owner therefore cannot retire, write down or reconcile the supply of an expired token: totalSupply and totalBurned freeze at their pre-expiry values permanently. The PoC confirms burn still reverts 3 650 days after expiry.

docs/SPEC.md documents burn purely in terms of InsufficientBalance, and says expiry blocks "every transfer (including mint)" — it never says burn is blocked. Holder balances becoming immovable at expiry is documented intent and is not a finding; the burn block is an unstated consequence of routing burns through the same hook.

Recommendation

Exempt burns from the expiry gate by using the hook's parameters:

```
function _beforeTokenTransfer(address, address to, uint256) internal virtual override {
    if (to != address(0) && ...) revert TokenExpired();
}
```

Or state in SPEC.md that burning after expiry is intentionally impossible.

M-21 Every GiftableToken holder carries an undocumented, unrevocable infinite allowance to Permit2

Severity	Medium
Target	src/GiftableToken.sol L10 (inherits lib/solady/src/tokens/ERC20.sol)
Proof	test_GT_permit2HasImplicitInfiniteAllowanceOverEveryHolder, test_GT_permit2AddressCanDrainHoldersWithoutApproval, test_GT_approveRevertsForPermit2AndCannotBeRevoked (in AuditPoCToken.t.sol)

Description

Solady's ERC20 enables Permit2 support by default — its own comment reads "Enabled by default. To disable, override `_givePermit2InfiniteAllowance()`" — and `GiftableToken` does not override it. Verified: no reference to Permit2 exists anywhere in `src/`.

The consequences apply to every holder of every voucher issued by this protocol:

- `allowance(anyHolder, 0x000000000022D473030F116dDEE9F6B43aC78BA3)` returns `type(uint256).max` for addresses that have never approved anything.
- `transferFrom` skips the allowance check entirely when the caller is Permit2, and does not decrement.
- `approve(PERMIT2, x)` for any `x != type(uint256).max` **reverts** with `Permit2AllowanceIsFixedAtInfinity`, so the allowance cannot be revoked and a standards-conformant integrator hits an unexpected revert.

`docs/SPEC.md` never mentions Permit2, so this is an undocumented protocol-wide trust assumption covering the entire voucher supply. On Celo mainnet the canonical Permit2 address holds the genuine Uniswap contract, which still requires an EIP-712 signature from the holder — so this is not a live drain today. It is rated Medium rather than higher for that reason. But the on-chain approve step is bypassed for every holder, the assumption is invisible to anyone reading the contract or the spec, and on any chain where that address holds different code it becomes a direct drain of every balance.

Recommendation

Decide explicitly. If Permit2 integration is not intended:

```
function _givePermit2InfiniteAllowance() internal view virtual override returns (bool) {  
    return false;  
}
```

If it is intended, document it in `SPEC.md` as a trusted external dependency, name the address, and note that the allowance cannot be revoked.

Low severity

L-1 With feeAddress == address(0) the pool fee is charged but never recorded

Severity	Low
Target	src/SwapPool.sol L313-315
Proof	test_POC_F07_zeroFeeAddress_confiscatesFeeSilently

Description

totalFee is always subtracted from the user's output, but crediting it is conditional:

```
uint256 netValue = quotedValue - totalFee - protocolFee; // always deducted
...
if (totalFee > 0 && feeAddress != address(0)) {
    fees[_outToken] += totalFee; // conditionally recorded
}
```

initialize accepts feeAddress == address(0) and test_getFee_no_policy shows that combination is expected. With a non-zero feePolicy and a zero feeAddress, the PoC user pays the full 1 % but fees[tokenOut] stays at zero: the amount is silently absorbed into pool liquidity, attributable to nobody and invisible in the accounting. Users are charged a fee no one has claimed.

Recommendation

Make the intent explicit. Either treat a zero feeAddress as a zero pool fee so users are not charged:

```
function getFee(address _inToken, address _outToken, uint256 _value) public view returns (uint256) {
    if (feePolicy == address(0) || feeAddress == address(0)) return 0;
    ...
}
```

or record the fee unconditionally and let withdraw be the only place that requires a non-zero feeAddress. Whichever is chosen, document it — the current behaviour is neither.

L-2 In coupled mode accrued fees are re-lent as swap liquidity

Severity	Low
Target	src/SwapPool.sol L292-298, L336-349
Proof	test_POC_F08_coupledMode_feesOverAccrue

Description

With feesDecoupled == false (the default) the liquidity check ignores fees[]:

```
if (IERC20(_outToken).balanceOf(address(this)) < quotedValue) revert InsufficientBalance();
```

Accrued fees are part of the balance and are therefore available to fund later swaps, while fees[_outToken] keeps incrementing. fees[] becomes a claim the pool cannot honour. The PoC drains a 100-token pool down to its fee reserve, swaps 1 more token out of that reserve, and ends with fees[tokenOut] at 1.01e18 against a balance of 0.01e18. withdraw(tokenOut) — the collect-all overload, which sets fees = 0 and then transfers the full recorded amount — reverts permanently.

The partial overload withdraw(tokenOut, value) still works for any value within the balance, so the owner is not locked out; they are simply owed more than the pool holds. docs/SPEC.md says fees "remain part of the pool liquidity" in this mode, so the lending is intended — what is not intended is fees[] recording an unbacked claim.

Recommendation

Either reserve fees in both modes by using the decoupled check unconditionally, or stop tracking fees[] when

feesDecoupled == false and have withdraw(token) collect against the balance. At minimum, cap collection at what is actually held:

```
uint256 balance = fees[_outToken];
uint256 held = IERC20(_outToken).balanceOf(address(this));
if (balance > held) balance = held;
```

L-3 Oracle freshness checks are incomplete and fail with an unattributable panic

Severity	Low
Target	src/OracleQuoter.sol L23, L98-121
Proof	test_POC_L4_updatedAtInFuture_panicsInsteadOfNamedError, test_POC_L7_unboundedStaleness_acceptsNeverUpdatedFeed

Description

Three gaps in the oracle read path, separate from the structural problem in M-4:

1. **DEFAULT_MAX_STALENESS = 86400.** A quoter initialised and left at the default trades on prices up to 24 hours old. Chainlink heartbeats are typically minutes to an hour, so the default is permissive by one to two orders of magnitude for most feeds.
1. **updatedAt == 0 is not checked explicitly.** All round metadata is destructured away:

“solidity returns (uint80, int256 answer, uint256, uint256 updatedAt, uint80) “

A round that never completed is rejected only as a side effect — `block.timestamp - 0 > maxStaleness` happens to be true. The protection therefore vanishes the moment `maxStaleness` exceeds the current block timestamp, and `test_POC_L7_unboundedStaleness_acceptsNeverUpdatedFeed` confirms that after `setMaxStaleness(type(uint256).max)` a feed reporting `updatedAt == 0` is priced normally. An operator “disabling staleness” unknowingly disables the empty-round guard too.

1. **A future updatedAt panics rather than reverting with a named error.** `block.timestamp - updatedAt` underflows if a feed reports a future timestamp, which is reachable with a misbehaving adapter or L2 timestamp skew. The direction is fail-closed, so it is not a loss — but the subtraction sits inside the try block’s **success** branch, which Solidity does not route to either catch clause. Callers get a raw `Panic(0x11)` instead of `StaleOraclePrice` or `OracleCallFailed`, so the contract’s error model cannot attribute the failure to the oracle at all. Related to I-5.

No L2 sequencer-uptime check. Chainlink’s current guidance is that consumers on L2 networks must read the sequencer uptime feed before trusting a price, because after a sequencer outage the aggregator resumes with an answer that looks fresh but was unpublishable during the downtime. `docs/DEPLOY.md` targets Celo mainnet (chain ID 42220), which now runs as an OP-Stack L2, so this applies — though the operator should confirm whether an uptime feed is published for their chain, since Chainlink’s own examples name Arbitrum, OP and Metis.

Recommendation

Derive the default from the heartbeats of the feeds actually in use rather than a flat day, and validate freshness explicitly before the subtraction:

```
if (updatedAt == 0 || updatedAt > block.timestamp) revert InvalidOraclePrice(oracle);
if (block.timestamp - updatedAt > maxStaleness) revert StaleOraclePrice(oracle);
```

Add a sequencer-uptime check if one is available for the deployment chain.

Note that `answeredInRound` is **not** worth adding: it is retained only for backwards compatibility on current OCR aggregators, where it tracks `roundId`, so an `answeredInRound >= roundId` check is a no-op on modern feeds. There is also no per-feed min/max answer sanity bound, so an aggregator pinned at its internal circuit-breaker limit during a crash would be priced as-is — worth noting, though newer aggregators no longer expose those bounds.

L-4 getScale's 77-decimal bound is unreachable

Severity	Low
Target	src/OracleQuoter.sol L123-144
Proof	test_POC_F10_getScale_boundIsUnreachable

Description

getScale guards against oversized decimals:

```
function getScale(uint8 decimals) internal pure returns (uint256) {
    if (decimals > 77) revert InvalidDecimals(decimals);
    return 10 ** uint256(decimals);
}
```

But determineOutput multiplies three of those scales together in plain 256-bit arithmetic *before* handing the result to fullMulDiv, which is the only part that widens to 512 bits:

```
FixedPointMathLib.fullMulDiv(inputValue, inRate * outScale * outRateScale, inRateScale * inScale * outRate);
```

With an 8-decimal feed (`inRate ~= 1e8`, `outRateScale = 1e8`), the numerator overflows once the token exceeds roughly 61 decimals. The PoC uses a 70-decimal token — accepted by `getScale` — and gets `Panic(0x11)` instead of `InvalidDecimals(70)`. The guard can never fire for the values it was written to catch.

Realistic decimals (6, 8, 18) are far from the boundary, so this is a robustness and error-clarity issue rather than an exploitable one.

Recommendation

Bound decimals where they can actually be honoured — 36 leaves ample headroom for any real token against an 18-decimal feed — and validate the feed's `decimals()` in `setOracle` so the failure surfaces at configuration time rather than at quote time.

L-5 A quote that truncates to zero consumes the input and returns nothing

Severity	Low
Target	src/SwapPool.sol L286-318
Proof	test_POC_F11_zeroQuoteConsumesInputForNothing

Description

`_swap` never checks that the swap produces a non-zero output. When `quotedValue` is 0 the whole body still succeeds: the liquidity check passes (`0 >= 0`), both fees are 0, `transfer(_recipient, 0)` returns true, and a Swap event is emitted with zeroes. The caller's input has already been taken by the deposit at the top of the function.

This is reachable with any quoter that scales down. `DecimalQuoter` (and `RelativeQuoter`, which divides before applying its rate) truncate an 18-decimal input to a 6-decimal output, so any amount below `1e12` wei quotes to zero. The PoC swaps `999_999_999_999` wei and the pool keeps all of it.

The per-call loss is bounded by just under one unit of the output token, so this is dust rather than a drain. It is worth fixing because the transaction reports success: a wallet or integration that sizes swaps from a user-entered amount has no signal that the trade was a total loss, and repeated small swaps accumulate.

Recommendation

Reject the degenerate case explicitly:

```
if (quotedValue == 0) revert InsufficientOutput();
```

Placing the check right after `getQuote` also covers the case where the whole output is consumed by fees. The `minAmountOut` parameter proposed in H-1 subsumes this for callers who use it, but the floor should not depend on

callers opting in.

L-6 maxStaleness has no fallback for zero, which is what an in-place upgrade reads

Severity	Low
Target	src/OracleQuoter.sol L30, L51-54, L106
Proof	test_POC_L7_maxStalenessZero_bricksQuoting, test_POC_L7_upgradeInPlace_readsZeroStaleness

Description

`maxStaleness == 0` means every quote reverts one second after any feed update — a total quoting DoS for every pool consuming the quoter. `setMaxStaleness` accepts 0 with no validation, and there is no `== 0` fallback at the read site.

That matters more than an owner typo, because zero is exactly what an in-place proxy upgrade reads. Confirmed against the repository history:

Commit	Storage layout
d909ed8	oracles @0, baseCurrency @1
14dda31	appends maxStaleness @2
b3e4b94	appends multiplier @3

A proxy initialised under d909ed8 and later upgraded in place to the current implementation has never written slot 2, so `maxStaleness` reads 0 and the quoter is dead on arrival. The PoC proves the layout with `vm.load` (slot 2 = 86400, slot 3 = 0), zeroes slot 2 with `vm.store`, and shows every quote reverting.

The asymmetry is the telling detail: the *later*-appended `multiplier` **does** have a zero fallback (`multiplier == 0 ? PPM : multiplier`, at both L94 and L171), and docs/SPEC.md documents it under an explicit “Upgrade note” saying “behaviour is preserved with no migration”. The identical hazard one slot earlier was never given the same treatment. Whether a d909ed8-era proxy is actually live is a deployment question, but the defensive inconsistency is a code fact.

Recoverable by the owner calling `setMaxStaleness`, hence Low.

Recommendation

Mirror the multiplier’s fallback and bound the setter at both ends:

```
uint256 effectiveStaleness = maxStaleness == 0 ? DEFAULT_MAX_STALENESS : maxStaleness;
...
function setMaxStaleness(uint256 _maxStaleness) public onlyOwner {
    if (_maxStaleness == 0 || _maxStaleness > MAX_ALLOWED_STALENESS) revert InvalidStaleness();
    ...
}
```

Rejecting 0 in the setter alone is not sufficient — the upgrade path never calls the setter. The read-site fallback is the part that matters. Extend docs/SPEC.md’s upgrade note to cover both appended variables.

L-7 Every proxied contract accepts owner == address(0) and consumes the initializer**Severity** Low**Target** all fourteen proxied contracts — SwapPool L84-110, OracleQuoter L43-49, RelativeQuoter L23-25, FeePolicy L30-36, ProtocolFeeController L30-44, Limiter L36-38, Splitter, GiftableToken L33-43, EthFaucet L40-43, PeriodSimple L24-27, CAT L33-35, AccountsIndex L34-37, ContractRegistry L24-30, TokenUniqueSymbolIndex L31-43**Proof** test_POC_L8_initializeOwnerZero_bricksQuoter, test_POC_L8_initializeOwnerZero_locksPoolAdmin; test_POC_initializeOwnerZero_bricksEveryProxiedContract (AuditPoCQuoter.t.sol); test_AI_initializeZeroOwner_leavesUnownedAndLiesAboutIsWriter, test_CR_initializeZeroOwner_bricksSetForever (AuditPoCIndex.t.sol); test_GT_initializeAcceptsZeroOwnerBrickingAdmin, test_CAT_initializeAcceptsZeroOwnerBrickingAdmin (AuditPoCToken.t.sol)**Description**

Not one of the fourteen proxied contracts checks its owner argument. Solady's `_initializeOwner` stores `address(0)` without complaint — `_guardInitializeOwner()` is left at its default `false` everywhere — the initializer modifier is then consumed, and the mistake is unfixable: `owner()` is zero, `msg.sender` can never equal it, and every `onlyOwner` path is dead forever. The instance must be abandoned and redeployed.

A side effect: in `GiftableToken`, `CAT` and `AccountsIndex`, `isWriter(address(0))` then returns `true`, because those implementations treat the owner as a writer. The accessor reports a writer that cannot exist.

These functions *do* validate their less important arguments. `OracleQuoter` rejects a zero `baseCurrency` — metadata that does not affect pricing — immediately before accepting a zero owner:

```
if (_baseCurrency == address(0)) revert InvalidBaseCurrency();
_initializeOwner(owner);
```

`ProtocolFeeController` rejects a zero `initialRecipient_` and a fee above PPM, then accepts a zero owner. `FeePolicy` validates `defaultFee_`, then accepts a zero owner. In each case the argument that controls everything is the one left unchecked.

Severity varies by what is lost, and in three cases the contract keeps working well enough to hide the problem:

- `FeePolicy`, `ProtocolFeeController`, `Limiter`, `RelativeQuoter`, `OracleQuoter`, `ContractRegistry` — configuration is frozen at its initial values. Recoverable by redeploying and repointing consumers.
- `GiftableToken` — no writer can ever be added, so a token deployed this way can never be minted at all. Fails obviously and early, which is the best outcome here.
- `CAT` — `addWriter` and `setTokensFor` are dead while `setTokens` still works, so users can self-serve but no administration is possible.
- `SwapPool` — worst, because it validates none of its nine address arguments and the failure is invisible. Swaps keep executing, so the pool looks healthy while every administrative path is permanently dead: accrued fees can never be collected, `withdrawLiquidity` can never rescue funds, and nothing can be sealed or reconfigured. Fees accumulate into a balance nobody can withdraw.
- `EthFaucet` — equivalent to a full seal. Configuration is frozen while claims keep flowing, which combines with [H-4](#) into a permanently ungated dispenser.
- `Splitter` — the escalation that turns [M-13](#) from a grieving nuisance into a permanent freeze: with no reachable owner, `updateSplit` can never route around a hostile recipient.

Recommendation

Reject a zero owner in all fourteen initializers. It is a one-line change per contract with no downside, and for `SwapPool` it converts a silent, permanent loss of administrative control into a failed deployment. Consider also rejecting zero for `SwapPool`'s `feePolicy/quoter/protocolFeeController` where the deployment does not intend the "unset" behaviour — the ungated-pool drain in [C-2](#) is the cost of treating `address(0)` as a valid configuration everywhere.

L-8 The registry check accepts any over-long reply and cannot report a codeless registry

Severity	Low
Target	src/SwapPool.sol L377-386
Proof	test_D1_fallbackRegistry_whitelistsEverything, test_D2_registryReturnShapes, test_D3_registryCallFailed_reachability (in AuditPoCSeal.t.sol)

Description

mustAllowedToken makes a raw call and checks only success, never the returned data length, and never probes that the target is a registry at all:

```
(bool r, bytes memory v) = _tokenRegistry.call(abi.encodeWithSignature("have(address)", _token));
if (!r) revert RegistryCallFailed();
bool isAllowed = abi.decode(v, (bool));
```

Measured behaviour across reply shapes:

Registry reply	Result
64 bytes, leading word true	accepted — trailing data ignored
abi.encode(uint256(2))	reverts (the decoder does validate bool cleanliness)
dirty high bits, low byte 1	reverts
31 bytes	reverts, empty revert data
address with no code	success == true, then decode reverts with empty data

Two consequences. Any contract with a permissive fallback that returns `abi.encode(true)` whitelists everything: the PoC installs one and drains the pool with an unregistered token. And `RegistryCallFailed` only fires when the registry *actively reverts*, never for a codeless or wrong-ABI address — the operator's most likely mistake produces an undiagnosable bare revert instead of the named error written for exactly that case. Same mechanism as I-5.

The wrong-address case is realistic rather than hypothetical because three unrelated contracts in this repository expose a matching `have(address)` signature — `AccountsIndex`, `TokenUniqueSymbolIndex` and `PeriodSimple`. Pointing `tokenRegistry` at the wrong one type-checks, calls successfully, and silently redefines the whitelist. `setTokenRegistry` is also unsealable (H-2) and accepts `address(0)`.

The call also uses `.call` rather than `staticcall`, handing the registry full reentrancy context. It is not exploitable in the current ordering — the re-entry point precedes every balance read and the `transferFrom` — but it is a gratuitous capability, and the sibling limiter check compiles to `STATICCALL` purely because `ILimiter.limitOf` happens to be declared `view`.

`RelativeQuoter.valueFor` and `reverseValueFor` have the identical pattern, reaching each token with `_token.call(abi.encodeWithSignature("decimals()"))` (L52-58, L98-104). `test_POC_relativeQuoter_decimalsIsNotStaticcall` proves it is not static by having the token write to its own storage while being read. That matters more here than in the registry case, because `tokenOut` is attacker-chosen and unvalidated (M-3), so reading a *decimals* getter hands an arbitrary address control inside `SwapPool._swap`, before the liquidity check. It is still not exploitable — the nested call performs its own checks and no state is stale at that point — but reading a `view` function should not be an execution opportunity at all.

Recommendation

Require an exact reply shape and use a typed static call:

```
(bool r, bytes memory v) = _tokenRegistry.staticcall(abi.encodeWithSignature("have(address)", _token));
if (!r || v.length != 32) revert RegistryCallFailed();
if (!abi.decode(v, (bool))) revert UnauthorizedToken();
```

Add an ERC-165 or sentinel probe in `setTokenRegistry` so a wrong address fails at configuration time rather than silently changing policy.

L-9 protocolFeeController is write-once and unsealable, so an unreceivable recipient bricks every swap

Severity	Low
Target	src/SwapPool.sol L36, L84-110, L308-310, L320-323
Proof	test_H5_unreceivableProtocolRecipient_bricksAllSwaps (in AuditPoCSeal.t.sol)

Description

The protocol-fee transfer is unconditional and precedes the user's:

```
if (protocolFee > 0) {
    if (!IERC20(_outToken).transfer(_getProtocolRecipient(), protocolFee)) revert TransferFailed();
}
```

If the recipient becomes unable to receive tokenOut — a blacklisting stablecoin, a contract that starts reverting, a recipient set to a rejecting address — every swap of that tokenOut reverts, and the pool has no lever to recover. protocolFeeController is written once in initialize and has no setter, sealable or otherwise. The PoC runs a successful swap, blacklists the protocol recipient in the token, and shows the next identical swap reverting; setProtocolFeeController does not exist to call.

Recovery depends entirely on the ProtocolFeeController owner — a different principal — calling setActive(false) or changing the recipient. A pool operator whose pools have stopped working has no unilateral remedy other than withdrawLiquidity and redeployment.

Recommendation

Add an owner-settable, sealable setProtocolFeeController, so a pool is not permanently coupled to a dependency it cannot replace. Alternatively make the protocol-fee transfer non-fatal — credit it to fees[] or forgo it when the transfer fails — so a broken recipient degrades the protocol's revenue rather than the pool's availability.

L-10 FeePolicy cannot express a zero fee for a pair, and its event says otherwise

Severity	Low
Target	src/FeePolicy.sol L38-44, L62-79, L87-96
Proof	test_POC_feePolicy_zeroPairFee_silentlyBecomesDefault, test_POC_feePolicy_zeroPairFee_eventContradictsState (in AuditPoCQuoter.t.sol)

Description

A stored pair fee of 0 is indistinguishable from no override, because every reader treats zero as "fall back":

```
uint256 pairFee = pairFees[pairKey];
return pairFee != 0 ? pairFee : defaultFee;
```

So an operator running a 10 % default who wants one pair to trade free calls setPairFee(A, B, 0) — which is accepted, passes every validation, and emits a success event — and that pair then charges 10 %. removePairFee is not an escape; it clears the override and lands on the same default. There is no way to express a free pair while any non-zero default exists, and the only workaround is to set defaultFee = 0 and then enumerate an explicit fee for every other pair, which inverts the point of having a default.

The event makes this worse rather than surfacing it:

```
emit PairFeeUpdated(tokenIn, tokenOut, oldFee, fee_); // fee_ == 0
```

The log states the new fee is 0 while getFee returns 100_000. An indexer or dashboard reconstructing fee history from events reports the pair as free when users are being charged the default — so the misconfiguration is invisible from exactly the place an operator would check.

docs/SPEC.md documents the fallback behaviour, so the mechanism is known. What is not addressed is that the setter accepts an input it silently ignores, and the event affirms it.

Recommendation

Make the sentinel explicit so zero is representable and the setter cannot lie. Either store `fee + 1` internally, or add a presence flag:

```
struct PairFee { bool isSet; uint256 ppm; }
mapping(bytes32 => PairFee) private pairFees;
...
PairFee memory p = pairFees[pairKey];
return p.isSet ? p.ppm : defaultFee;
```

If the fallback semantics must be preserved for storage-layout reasons, at minimum reject `fee_ == 0` in `setPairFee` with a named error pointing the caller at `removePairFee`, so the operator learns the value was not applied.

Note that the pair key itself is sound: `keccak256(abi.encodePacked(tokenIn, tokenOut))` packs two fixed-width 20-byte addresses, so no collision is possible and direction remains independent (`test_feePolicy_pairKey_hasNoCollision_holds`).

L-11 AccountsIndex.time() leaks the blocked flag into the returned timestamp

Severity	Low
Target	src/AccountsIndex.sol L17, L64-68, L118-121
Proof	test_AI_time_leaksBlockedFlagIntoTimestamp, test_AI_packedLayout_timestampFieldOverlapsBlockedFieldBit (in AuditPoCIndex.t.sol)

Description

`entryIndex` packs three fields into one word, but the field widths are not enforced and two of them overlap:

Bits	Field	Written by	Read by
63..0	1-based <code>entryList</code> position	<code>uint64(i)</code>	<code>uint64(...)</code> ☒
255..64	<code>block.timestamp</code> , unclamped	<code>block.timestamp << 64</code>	<code>>> 64 ×</code>
128	<code>BLOCKED_FIELD</code>	<code>'\</code>	<code>= 1 << 128'</code> & <code>BLOCKED_FIELD</code> ☒

`BLOCKED_FIELD` sits at bit 128, inside the region `time()` shifts down, and `time()` applies no mask:

```
return entryIndex[_account] >> 64;
```

For a deactivated account the stored word is `idx | (ts << 64) | (1 << 128)`, so `time()` returns `ts + 2**64` — a timestamp roughly 584 billion years in the future, in place of the “block timestamp when the account was added” that `docs/SPEC.md` documents. The corruption appears on `deactivate()` and silently disappears on `activate()`, so it is state-dependent rather than persistent, which is what makes it easy to miss.

No contract in `src/` consumes `time()` today, which caps this at Low. But any off-chain age gate, vesting schedule or rate limiter built on it would be defeated for exactly the accounts an operator has flagged — the population where correctness matters most.

The same missing clamp means a timestamp of `2**64` or more would set bit 128 directly from `add()`, so an entry would be born blocked. That is unreachable on any real chain and is noted only as evidence that the widths are enforced nowhere.

Recommendation

Clamp on write and mask on read so the two agree, and move the flag out of the timestamp’s reach:

```
uint256 constant BLOCKED_FIELD = 1 << 255;
...
_entry |= uint256(uint64(block.timestamp)) << 64; // write
return (entryIndex[_account] >> 64) & type(uint64).max; // read
```

Document the layout as `[255] = blocked | [127:64] = uint64 timestamp | [63:0] = uint64 index`.

L-12 AccountsIndex.add(address(0)) whitelists the null address

Severity	Low
Target	src/AccountsIndex.sol L59-72
Proof	test_AI_add_zeroAddress_isWhitelistedAndAliasesTheSentinel (in AuditPoCIndex.t.sol)

Description

entryList[0] = address(0) is a reserved sentinel that makes index 0 mean "absent". add() has no zero-address guard, so add(address(0)) passes the entryIndex[_account] != 0 check, pushes address(0) a second time, and have(address(0)) and isActive(address(0)) both begin returning true.

Because have() is the security boundary consumed by SwapPool.mustAllowedToken and EthFaucet's registry check, the null address becomes an allowed token and an allowed faucet recipient. Independently, entry(i) can now return address(0), which every consumer reads as "unset", so enumeration becomes ambiguous.

Requires owner or writer, hence Low. ContractRegistry already has a ZeroAddress error for exactly this class of input.

Recommendation

if (_account == address(0)) revert ZeroAddress(); at the top of add().

L-13 A bytes32-returning symbol() cannot be indexed

Severity	Low
Target	src/TokenUniqueSymbolIndex.sol L57-62, L83-88
Proof	test_TI_bytes32SymbolToken_cannotBeRegistered; test_TI_stringSymbolDecodesCorrectly_isCorrect, test_TI_symbolBoundary32Bytes_noSilentTruncation_isCorrect (in AuditPoCIndex.t.sol)

Description

register() and add() decode the symbol as a dynamic type:

```
(bool ok, bytes memory r) = _token.call(abi.encodeWithSignature("symbol()"));
if (!ok) revert();
tokenSymbol = abi.decode(r, (bytes));
```

For a string-returning symbol() this is correct — string and bytes share an identical ABI encoding — and the subsequent bytes32(tokenSymbol) does not truncate, because if (tokenSymbol.length > 32) revert TokenSymbolTooLong() guards it. Both properties are pinned by tests, which also settles the unsafe-typecast warning forge lint raises on that cast: it is a false positive.

The gap is the MKR-style token that returns bytes32. Its 32 bytes are *static* data, so abi.decode reads the symbol itself as an ABI head offset and reverts. Such tokens can never be indexed, and the failure surfaces as a bare panic with no actionable error. Two related roughnesses: if (!ok) revert() discards the callee's revert data, and r.length is never checked before decoding.

Recommendation

Branch on the return length and name the error:

```
if (!ok) revert SymbolCallFailed();
bytes32 symbolKey;
if (r.length == 32) {
    symbolKey = abi.decode(r, (bytes32)); // MKR-style
} else {
    bytes memory s = abi.decode(r, (bytes)); // string / bytes
    if (s.length > 32) revert TokenSymbolTooLong();
```

```

    symbolKey = bytes32(s);
}

```

L-14 TokenUniqueSymbolIndex.initialize does not enforce equal array lengths

Severity	Low
Target	src/TokenUniqueSymbolIndex.sol L31-43; cmd/ge-publish/main.go L409
Proof	test_TI_initializeArrayLengthMismatch_isUnchecked (in AuditPoCIndex.t.sol)

Description

docs/SPEC.md states the two arrays “must be the same length”, but nothing checks it. The loop is bounded by `initialTokens.length`, so surplus symbols are silently dropped and a short symbol list reverts with a bare `Panic(0x32)` instead of a named error.

The publisher’s guard is also incomplete:

```

if len(indexTokens) > 0 && len(indexSymbols) > 0 && len(indexTokens) != len(indexSymbols) {

```

The comparison is skipped whenever *either* list is empty, so passing tokens with no symbols clears the CLI check and then panics inside the constructor call.

Recommendation

if (`initialTokens.length != initialSymbols.length`) revert `LengthMismatch()`; in the contract, and drop the `len(...) > 0` conjuncts from the Go guard so it checks the lengths unconditionally.

L-15 ContractRegistry.initialize accepts duplicate identifiers

Severity	Low
Target	src/ContractRegistry.sol L24-30
Proof	test_CR_duplicateIdentifiersInInitialize_inflateCountAndAliasSlots, test_CR_initializeWithNoIdentifiers_isPermanentlyUnusable (in AuditPoCIndex.t.sol)

Description

The identifier list is pushed blindly, so `[KEY_A, KEY_A, KEY_B]` gives `identifierCount() == 3` with `identifier(0) == identifier(1)`. Since set is write-once per key, only two of the three advertised slots can ever hold an address; the duplicate is dead weight that advertises a configuration slot which can never be independently filled.

The identifier set is fixed at initialization with no way to add or remove keys, so a deployment made with a mis-generated list — a repeat in `--registry-identifiers` — can only be corrected by redeploying. An empty list is accepted the same way and yields a registry where set always reverts `IdentifierNotFound`; the Go CLI guards that case but the contract does not.

Everything else about this contract is correct: write-once enforcement, unknown identifier rejection, zero-address rejection, `onlyOwner` on set, and `identifier(idx)` bounds all behave as specified.

Recommendation

De-duplicate in initialize — an $O(n^2)$ scan is fine for a deploy-time list — and reject an empty list:

```

if (_identifiers.length == 0) revert IdentifierNotFound();
for (uint256 j = 0; j < identifier.length; j++) {
    if (identifier[j] == _identifiers[i]) revert IdentifierAlreadyExists();
}

```

L-16 Splitter validation is $O(n^2)$ and re-runs on every distribution, with no cap on recipients

Severity	Low
Target	src/Splitter.sol L88-97
Proof	test_POC_splitter_quadraticValidation_exceedsBlockGasLimit (in AuditPoCFunds.t.sol)

Description

The duplicate scan is a complete nested loop over all $n(n-1)/2$ index pairs, and `_validateSplit` runs on *every* `distributeETH` / `distributeERC20` call, not only when the split is set. Nothing caps `accounts.length`.

Measured gas for validation and hashing alone, before any transfer:

Recipients	Gas
50	690 416
100	2 708 367
200	10 734 853
400	42 750 170

The cost is cleanly quadratic and crosses a 30M block gas limit at roughly $n \approx 340$, at which point the split can never be distributed again. Recovery depends on the owner shrinking it, which reintroduces the [M-13](#) and [M-15](#) dependence on the owner being reachable and honest.

Re-deriving membership properties on the distribution path buys nothing: the hash already binds the arrays to what was validated when the split was set.

Recommendation

Cap `accounts.length` (64 is generous for a payment split), and run the duplicate check only in `initialize` / `updateSplit`. Requiring accounts to be strictly ascending would make duplicate detection $O(n)$ and remove the quadratic term entirely.

L-17 Splitter.getHash uses abi.encodePacked on two dynamic arrays; the collision is real and unreachable only by accident

Severity	Low (latent)
Target	src/Splitter.sol L74-76 (mitigated by the ordering at L45-46, L57-58)
Proof	test_POC_splitter_getHashCollisionExists_butIsUnreachable, test_splitter_hashInjectiveOverValidatedDomain_holds (in AuditPoCFunds.t.sol)

Description

```
return keccak256(abi.encodePacked(accounts, percentAllocations));
```

Two adjacent dynamic arrays with no length prefix and no delimiter is the textbook hash-collision anti-pattern, and here it does produce genuine collisions. `abi.encodePacked` pads *array elements* to 32 bytes, so a `uint32` allocation left-padded to 32 bytes is byte-identical to `address(uint160(thatValue))` left-padded to 32 bytes. An allocation can therefore be migrated across the array boundary as an “address” leaving the preimage unchanged:

```
(accounts = [r1, r2], allocs = [600_000, 400_000])
(accounts = [r1, r2, address(uint160(600_000))], allocs = [400_000])
```

Both hash to the same value, for arbitrary real recipient addresses.

What stops exploitation is that `_validateSplit` runs **before** `_validateHash` on all four entry points and enforces `accounts.length == percentAllocations.length`. Every accepted pair therefore has $n == m$, the total encoding

length is exactly $64n$, the boundary is pinned at $32n$, and digest equality forces element-wise equality. The colliding pair reverts `AccountsAndAllocationsMismatch` before the hash is ever compared.

So `_hashSplit` is injective over the validated domain — but only as a side effect of the order of two statements, in a commitment that governs custody of funds. Swapping those two lines during a future refactor would silently make a forged split spendable, and nothing in the test suite pins the ordering.

Recommendation

`keccak256(abi.encode(accounts, percentAllocations))`. It costs nothing and removes the dependence on statement order entirely. Note this changes the value `getHash()` returns, so it must ship together with any off-chain tooling that recomputes it, and with `docs/SPEC.md`'s description of the commitment.

L-18 RescueVault's payable catch-all fallback reports success for every unknown selector

Severity	Low
Target	<code>src/RescueVault.sol</code> L46
Proof	<code>test_POC_rescueVault_fallbackSilentlySucceedsForAnyCall</code> (in <code>AuditPoCFunds.t.sol</code>)

Description

The vault's stated purpose is to occupy a `CREATE` address that another contract was expected to take — an address third parties will keep calling. `fallback() external payable {}` accepts any calldata, keeps any attached ETH, and returns success with empty returndata.

Anything using a low-level `call` and checking only success — routers, multicall batchers, scripts, front-ends — therefore concludes that a `deposit()`, `stake()`, `mint()` or `transfer()` against that address worked. The value is at least recoverable by admin, which is the point of the vault, but the acknowledgement is indistinguishable from a real one, so misdirection is masked rather than surfaced. High-level Solidity callers fare only marginally better: they revert while decoding empty returndata, producing a confusing failure instead of a clear "no such function".

`test/RescueVault.t.sol`'s `test_receiveAndFallback_acceptETH` asserts this as desired behaviour — it sends `hex"deadbeef"` with value and requires success — so the hazard is currently pinned in as a feature.

Recommendation

Delete `fallback()` and keep only `receive() external payable {}`. Bare ETH transfers still work and every unknown selector reverts, which is the correct signal from a squatted address. If a fallback must stay for compatibility, make it non-payable and emit an event so misdirected calls are at least observable.

L-19 RescueVault.admin is immutable with no rotation path, and rotation attempts fake success

Severity	Low
Target	<code>src/RescueVault.sol</code> L28, L39-42 (compounded by L46)
Proof	<code>test_POC_rescueVault_adminIsImmutableButRotationCallsFakeSuccess</code> (in <code>AuditPoCFunds.t.sol</code>)

Description

`admin` is set once in the constructor with no transfer, handover or renounce. Losing the key permanently strands every asset that ever lands in the vault; compromising it hands over every asset, present and future, with no recovery. For a contract whose whole job is custody of misdirected funds — and which may sit at its address indefinitely — a single unrotatable key is a strong assumption.

L-18 makes it actively misleading. An operator attempting `setAdmin(address)` or `transferOwnership(address)` through a raw call gets `success == true` from the fallback while `admin()` is unchanged, so a rotation can be believed to have happened. The PoC shows both calls succeeding and the intended new admin then being rejected with `Unauthorized`.

Recommendation

Make admin a mutable storage variable with a two-step handover — solady's Ownable provides this and costs one SLOAD per sweep — and fix fallback() per L-18 so a mistaken rotation call reverts loudly.

L-20 RescueVault batch sweeps are all-or-nothing, and duplicate ids revert them

Severity	Low
Target	src/RescueVault.sol L66-73, L81-85, L94-106
Proof	test_POC_rescueVault_erc1155BatchDuplicateId_revertsWholeSweep, test_POC_rescueVault_erc721BatchDuplicateId_revertsWholeSweep, test_POC_rescueVault_oneBadTokenBlocksWholeErc20Batch, test_POC_rescueVault_codelessTokenInBatch_reverts (in AuditPoCFunds.t.sol)

Description

All three batch helpers are unbounded single-transaction loops in which one failing element reverts the whole call, so a rescue of N assets is only as reliable as its worst element. One blacklisting or paused ERC20, one address typo that is not a contract, or one token id the vault does not actually own kills the batch and nothing is rescued.

sweepERC1155Batch has an additional defect: amounts are read from balanceOf for **all** ids first — correct for distinct ids — and then a single safeBatchTransferFrom is issued, so a repeated id is counted twice and the batch requests more than the vault holds. sweepERC721s fails the same way on a repeated id, because the vault no longer owns it on the second pass.

Token lists and ids are admin-supplied, so this is not griefing. It matters because a rescue is precisely the situation where an operator is working from an imperfect list under time pressure, and the per-item functions still work — so the batch helpers are strictly more fragile than calling them in a loop off-chain.

Recommendation

De-duplicate (or require strictly ascending ids) in sweepERC1155Batch and sweepERC721s, and add best-effort variants that wrap each element in a low-level call, skip failures, and return a per-element success array so a partial rescue succeeds.

L-21 GiftableToken allows mint and transfer to address(0)

Severity	Low
Target	src/GiftableToken.sol L80-84
Proof	test_GT_mintToZeroAddressInflatesSupplyIrrecoverably, test_GT_transferToZeroAddressSilentlyDestroysTokens (in AuditPoCToken.t.sol)

Solady's ERC20 header states plainly that it will not revert on zero-address mint or transfer and instructs integrators to add their own checks. GiftableToken adds none.

mintTo(address(0), n) credits the zero address and inflates totalSupply and totalMinted with tokens nobody can move, since burn only burns from msg.sender. transfer(address(0), n) destroys a holder's tokens while leaving totalSupply unchanged and totalBurned at zero — so circulating supply is silently overstated for any consumer computing it from totalSupply or totalMinted - totalBurned. The PoC mints 1 000, sends 400 to the zero address, and shows totalSupply still reporting 1 000 with totalBurned at 0.

Recommendation: reject address(0) in mintTo, and guard it for non-burn transfers in _beforeTokenTransfer.

L-22 Unbounded setPeriod overflows lastUsed + period and selectively bricks returning claimants

Severity	Low
Target	src/PeriodSimple.sol L29-32, L43-45, L47-55
Proof	test_PS_unboundedPeriodOverflowBricksPreviouslyPokedSubjects (in AuditPoCToken.t.sol)

setPeriod accepts any uint256, and next() computes lastUsed[_subject] + period in checked arithmetic. have() reaches it through the external self-call this.next(_subject), so the panic is bubbled up rather than handled.

With a period near 2^{256} , next(), have() and therefore poke() all revert Panic(0x11) for every subject with lastUsed != 0, which the faucet surfaces as PeriodBackend on gimme, giveTo **and** check — the last of which is specified to return a bool. Subjects who have never claimed take the lastUsed == 0 early return and keep working, so the breakage is selective and invisible until a returning user complains.

Owner-triggerable and recoverable by another setPeriod, hence Low. The external-self-call pattern is what turns a comparison into a hard failure instead of a false.

Recommendation: bound the period, and compare without overflow:

```
return block.timestamp > lastUsed[_subject] && block.timestamp - lastUsed[_subject] > period;
```

Convert this.have(...) / this.next(...) to internal calls.

L-23 A poke consumes a subject's cooldown with no payout, and cannot be undone

Severity	Low
Target	src/PeriodSimple.sol L57-64
Proof	test_PS_pokerCanBurnSubjectCooldownWithNoPayoutAndNoReset (in AuditPoCToken.t.sol)

poke is callable directly by owner() or poker and only records usage — it dispenses nothing. Either privileged key can therefore burn any address's claim slot for a full period without paying it, and there is no administrative reset: no resetLastUsed, unpoke, clearLastUsed or setLastUsed exists. The PoC probes four candidate reset selectors, all of which fail.

The realistic harm is operational rather than adversarial: a batch poke run against the wrong address list locks out every affected user for a full period with no remedy short of globally changing period.

Recommendation: restrict poke to the poker alone — the faucet is the only legitimate caller — and add resetLastUsed(address) external onlyOwner.

L-24 amount == 0 still consumes the recipient's cooldown

Severity	Low
Target	src/EthFaucet.sol L134-139
Proof	test_EF_zeroAmountClaimBurnsVictimCooldownForFree (in AuditPoCToken.t.sol)

Neither setAmount nor _checkAndPoke rejects a zero amount, and poke runs before the payout, so anyone can call giveTo(victim) while amount == 0 and consume the victim's entire cooldown while transferring nothing.

This matters because zeroing amount is the natural way to pause a faucet that has no pause flag. Doing so hands an attacker a free primitive to lock out every user for a full period — the PoC burns a victim's 1-day cooldown, and the victim's gimme() then reverts even after the owner restores the amount.

Recommendation: if (amount == 0) revert InvalidState(); in _checkAndPoke, and add an explicit pause flag rather than overloading amount = 0.

L-25 PeriodSimple.next() contradicts have() for a never-poked subject

Severity	Low
Target	src/PeriodSimple.sol L43-45
Proof	test_PS_nextContradictsHaveForNeverPokedSubject (in AuditPoCToken.t.sol)

next() returns lastUsed[_subject] + period, which for a never-poked subject is 0 + period — a relative duration returned where callers expect an absolute timestamp. have() says the subject is eligible now; next() says they must wait until period. With a large configured period this reads as a far-future timestamp, so any client gating its UI on nextTime(subject) <= block.timestamp tells brand-new users they are not yet eligible. EthFaucet.nextTime propagates the value verbatim.

The behaviour matches SPEC.md:492 literally, so this is a defect in the specification and the code together rather than a deviation.

Recommendation: return lastUsed[_subject] == 0 ? 0 : lastUsed[_subject] + period;, and mirror the change in SPEC.md.

L-26 nextTime reverts undecodably when unset, and nextBalance ignores its own argument

Severity	Low
Target	src/EthFaucet.sol L12, L141-151
Proof	test_EF_nextTimeWithUnsetPeriodCheckerRevertsUndecodably, test_EF_nextBalanceIgnoresItsSubjectArgument (in AuditPoCToken.t.sol)

nextTime calls periodChecker.call(...) with no zero-address guard. A call to address(0) succeeds with empty returndata, so the if (!ok) revert PeriodBackendError() guard is bypassed and the function dies inside abi.decode with an opaque revert instead of the dedicated error.

nextBalance is worse. It builds abi.encodeWithSignature("balanceThreshold()", _subject), appending 32 bytes of dead calldata to a zero-argument selector, and returns a *global* threshold as though it were subject-specific. The _subject parameter is entirely meaningless — the PoC shows nextBalance(user1) == nextBalance(user2) == nextBalance(address(0)). The function is not documented in SPEC.md at all.

Two smaller items in the same area: error NotOwner() is declared and never used, and PeriodBackend is reused for three semantically distinct failures (call failure, poke returning false, and _checkAndPoke returning false), so a caller cannot tell a broken backend from a cooldown that simply has not elapsed.

Recommendation: guard periodChecker == address(0) in both functions, validate result.length >= 32, drop the _subject parameter from nextBalance or delete the function, remove NotOwner, and split PeriodBackend into distinct errors.

L-27 The expiry timestamp has no getter, and initialize accepts an expiry already in the past

Severity	Low
Target	src/GiftableToken.sol L21, L33-43, L114-118
Proof	test_GT_expiryTimestampHasNoOnChainGetter, test_GT_initializeAcceptsPastExpiryProducingDeadToken (in AuditPoCToken.t.sol)

_expires is private with no accessor, so neither clients nor integrating contracts can read when a voucher expires — while supportsInterface answers true for seven interface ids. For a token whose entire purpose is time-limited value, the expiry being unreadable on-chain is a significant omission; the PoC probes five plausible getter signatures and none exist.

Separately, initialize accepts expiresAt < block.timestamp, producing a token that reverts TokenExpired on its very first mint. Because _expires is write-once, that deployment is dead on arrival with no way to fix it.

Recommendation: add function expires() public view returns (uint256), reject a past expiresAt in

initialize, and re-derive the supportsInterface ids from the functions actually implemented.

L-28 EthFaucet.giveTo(address(0)) burns ETH: there is no zero-recipient check

Severity	Low
Target	src/EthFaucet.sol L134-139
Proof	test_POC_L28_giveToZeroAddress_burnsEth_ungatedLoop, test_POC_L28_giveToZeroAddress_burnsEth_periodGated (in AuditPoCGaps.t.sol)

Description

giveTo validates nothing about its recipient — unlike SwapPool’s four-argument withdraw, which reverts InvalidRecipient for the zero address:

```
function giveTo(address _recipient) public returns (uint256) {
    if (!_checkAndPoke(_recipient)) revert PeriodBackend();
    payable(_recipient).transfer(amount);
    ...
}
```

payable(address(0)).transfer(amount) succeeds — the zero address is an ordinary account — so the payout is **destroyed, not delivered**. Two configurations make it reachable:

- **The ungated default (H-4):** no registry, no period checker, so _checkAndPoke returns true unconditionally and giveTo(address(0)) can be called in a loop. The PoC destroys a 5 ether balance in one transaction. This is distinct from H-4’s theft: the funds are destroyed rather than taken, so the attacker needs no recipient rotation and gains nothing.
- **A period-gated faucet** (periodChecker set, balanceThreshold == 0, no registry): PeriodSimple.poke(address(0)) records the cooldown against the null address and returns true, so the burn succeeds once per period — the PoC burns again after one warp. Here the cooldown the operator *did* configure provides no protection, because the only check that would reject the null address is the optional registry — and L-12 shows even that can be made to whitelist address(0).

The burn direction is what separates this from M-17: recipient rotation defeats the cooldown for *theft*, but rotation is unnecessary for *destruction* — only the zero address burns, and it needs no second identity.

Recommendation

```
if (_recipient == address(0)) revert InvalidRecipient();
```

Applying the same check to gimme is unnecessary (msg.sender cannot be zero), but giveTo should reject the null address outright, and H-4’s fail-closed registry recommendation is the deeper fix.

L-29 The limiter cap is checked against the pre-transfer balance, so a hooked tokenIn can nest deposits and exceed it

Severity	Low
Target	src/SwapPool.sol L168-174, L388-396
Proof	test_POC_L29_hookedTokenIn_nestedDeposit_exceedsLimiterCap (in AuditPoCGaps.t.sol)

Description

mustWithinLimit reads the pool’s balance **before** the transferFrom that the check is meant to bound:

```
uint256 limit = ILimiter(tokenLimiter).limitOf(_token, address(this));
uint256 balance = IERC20(_token).balanceOf(address(this));
if (balance + _valueDelta > limit) revert LimitExceeded();
```

```
// ... then IERC20(_token).transferFrom(msg.sender, address(this), _value)
```

A token with a tokens-to-send hook (ERC-777, ERC-1363, or any custom token that notifies the sender) fires inside `transferFrom` **before the pool is credited**. The hook — which any holder of a whitelisted hook-capable token can register for their own transfers via ERC-1820, without the token issuer’s involvement — nests a second deposit of the same token. The nested call re-reads the same pre-transfer balance and passes the same check:

```
cap = 1 000, balance = 0
outer deposit(600): check 0 + 600 <= 1 000 ok -> transferFrom starts
└─ hook: nested deposit(600): check 0 + 600 <= 1 000 ok -> completes, balance = 600
outer transferFrom completes -> balance = 1 200 > cap
```

The PoC ends with the pool holding 1.2× its configured cap, and further honest deposits reverting `LimitExceeded` — the documented invariant (“a deposit must not push the pool balance above the limiter cap”, `docs/SPEC.md`) is violated for as long as the excess stands. Nesting deeper multiplies the excess further, bounded only by the attacker’s balance.

This is a distinct reentrancy path from M-2: M-2 is a `tokenOut` hook firing *after* the outbound transfers to corrupt `fees[]`; this is a `tokenIn` hook firing *during* the inbound transfer to corrupt the exposure bound. The impact is bounded compared to M-2 — the attacker deposits real tokens and can only swap them out at the quoter’s rate, so the pool’s loss is realised only if the token is overvalued or later depegs — but the limiter is the control an operator sets to cap exactly that exposure, and it is defeated completely. It also matters disproportionately because the limiter is currently the only thing standing between an ungated pool and C-2.

Recommendation

The M-2 fix covers this path as well: a `nonReentrant` guard on `deposit` and both swap entry points makes the nested call impossible. Independently, the C-2 balance-delta fix closes it a second way — measure the balance after the transfer and revert `LimitExceeded` if the post-transfer balance exceeds the cap, rather than predicting it from the pre-transfer balance.

L-30 `ge-publish publish-one --contract swappool` rejects every quoter value; the command is dead

Severity	Low
Target	<code>cmd/ge-publish/main.go</code> L437-454
Proof	code trace — the three sequential validations are mutually exclusive

Description

The `publish-one` `swappool` path validates `--pool-quoter` three times in a row, and no input can survive all three:

```
if strings.EqualFold(cfg.PoolQuoter, "oracle") {
    quoterAddress, err = parseAddress(cfg.BaseCurrency) // parsed...
    ...
    _ = quoterAddress // ...and discarded
}
if cfg.PoolQuoter != "" && cfg.PoolQuoter != "relative" && cfg.PoolQuoter != "oracle" {
    return errors.New("pool-quoter must be relative|oracle")
}
if !common.IsHexAddress(cfg.PoolQuoter) {
    return errors.New("for publish-one swappool, pass quoter proxy address in --pool-quoter")
}
```

Input	Outcome
hex address (0x...)	fails the second check: "must be relative"
relative (the default)	fails the third check: not a hex address
oracle	parses --base-currency, discards it, then fails the hex check
empty	fails the hex check

Every branch errors, including the default value from `envOr("POOL_QUOTER", "relative")`, so `publish-one --contract swappool` can never deploy a pool. `docs/PUBLISH.md` states the CLI "expects a hex address for an already deployed quoter proxy" — that is true only of `deploy-proxy` (`encodeInitFor`, L896-899); the `publish-one` path rejects hex addresses too. `docs/DEPLOY.md`'s main flow uses `deploy-proxy` and is unaffected, which is why the dead path went unnoticed.

Recommendation

Delete the `relative|oracle` branch and the dead `oracle` special-case, and accept only a hex address in the `publish-one` path, matching `encodeInitFor`. Until then, document that pools can only be deployed via `deploy-impl + deploy-proxy`.

L-31 Proxy salts are derived only from (deployer, contract name): one proxy per contract name per deployer per factory

Severity	Low
Target	<code>pkg/publish/publish.go</code> L229-243 (<code>GenerateSalt</code>); used at <code>cmd/ge-publish/main.go</code> L588, L789
Proof	code trace — the salt has no per-instance input

Description

```
salt := make([]byte, 32)
copy(salt[:20], deployer.Bytes())
copy(salt[20:], id) // first 12 bytes of the contract name
```

The proxy salt is a pure function of the deployer address and the contract name. A second `deploy-proxy --contract swappool` from the same key through the same factory produces the identical salt, `CREATE2` computes the identical address, and the deployment reverts `DeploymentFailed` because the address is occupied. There is no flag to vary it — `--factory-salt-suffix` affects only the factory's own deployment salt, not proxy salts.

One deployer key can therefore deploy exactly **one** `SwapPool`, **one** `GiftableToken`, one of each contract, per factory. `docs/DEPLOY.md` says "Multiple proxies can share one implementation" — true of the proxy pattern, but unreachable through this tool from a single key. For a protocol whose purpose is many community pools and vouchers, the publisher cannot deploy a second pool or token without rotating to a fresh deployer key (which also changes the factory address and every downstream deterministic address).

Third-party squatting is *not* possible: `ERC1967Factory` requires the salt to start with the caller's address, so only the deployer can use their own salt prefix. This is purely self-collision, but it is a hard operational wall that surfaces only on the second deployment, with an unattributed revert.

Recommendation

Mix a user-supplied nonce into the proxy salt (e.g. `--proxy-salt-suffix`, appended to the name before truncation), or derive the salt from a hash of the contract name and the init arguments so distinct configurations yield distinct addresses.

Informational

I-1 supportsInterface advertises incorrect interface IDs

Severity	Informational
Target	src/OracleQuoter.sol L198-209 (same pattern in src/RelativeQuoter.sol)
Proof	test_POC_I1_supportsInterface_reportsWrongIds

0xdbb21d40 is the raw function selector of `valueFor(address,address,uint256)`, not an ERC-165 interface ID. It is the legacy one-method "TokenQuote" ID, predating the addition of `reverseValueFor` in e0ff3d5. The true ID for the current two-method `IQuoter` is `valueFor ^ reverseValueFor = 0xdbb21d40 ^ 0x56558fc8 = 0x8de79288` — confirmed equal to `type(IQuoter).interfaceId` — and `supportsInterface(0x8de79288)` returns false. A consumer performing correct ERC-165 detection of `IQuoter` gets false from every implementation, and there is no way to detect whether a given quoter supports `reverseValueFor` — which `SwapPool.getAmountIn` calls unconditionally, so a pre-e0ff3d5 quoter left configured on a pool would revert every exact-output quote.

0x9493f8b2, commented "ERC173" in `RelativeQuoter`, is not the ERC-173 ID. The canonical value is 0x7f5828d0 (`owner() ^ transferOwnership(address)`), which `src/Limiter.sol` uses correctly. Both quoters advertise a wrong ID and omit the right one.

Recommendation: report 0x8de79288 for `IQuoter` and 0x7f5828d0 for ERC-173, keeping the legacy 0xdbb21d40 only if existing off-chain consumers depend on it, with a comment explaining that it is a selector and not an interface ID.

I-2 Three declared errors in SwapPool are never used

Severity	Informational
Target	src/SwapPool.sol L19 (<code>TokenCallFailed</code>), L21 (<code>QuoterCallFailed</code>), L26 (<code>LimiterCallFailed</code>)

All three are declared and never reverted; every other declared error has at least one revert site. They are not merely dead weight — they document an intent the code does not implement. `getQuote` and `mustWithinLimit` call the quoter and limiter through typed interfaces with no `try/catch`, so a failing quoter or limiter propagates the callee's own revert data and the caller cannot tell a pool-level failure from a dependency failure. Note the asymmetry: `mustAllowedToken` *does* wrap its registry call and reverts `RegistryCallFailed`, so two of the three external dependencies are handled inconsistently with the third.

Recommendation: either wire the errors up so dependency failures are attributable, or delete them.

I-3 SwapRouter does not validate path continuity

Severity	Informational
Target	src/SwapRouter.sol L19-36

Neither quote function checks that `path[i].tokenOut == path[i + 1].tokenIn`, nor that each pool is actually a `SwapPool`. A malformed path returns a plausible-looking number computed from unrelated hops. Since the router holds no funds and only reports amounts, the consequence is a wrong quote rather than a loss — but a caller acting on that quote will size a real swap from it.

Recommendation: add the continuity check to both loops. It is two comparisons and it turns a silently wrong answer into a revert.

I-4 Seal state-machine rough edges

Severity	Informational
Target	src/SwapPool.sol L124-138, L336-362
Proof	test_POC_I04_sealStateMachineEdges; test_A1_seal_zero_isRepeatableNoop, test_A2_isSealed_sevenReverts, test_A3_seal_partialOverlapReverts, test_H3_zeroValueFeeCollect (in AuditPoCSeal.t.sol)

The substantive seal problems are in [H-2](#). These are the interface edges that remain:

- **isSealed rejects its own maximum.** if (`_state >= maxSealState`) revert `InvalidState()`, so `isSealed(7)` reverts even though `seal(7)` is legal — the domain accepts every mask except the one that names everything, and callers must know to pass 0 instead. `docs/SPEC.md` documents the workaround, but integrators will hit it. Use `_state > maxSealState`.
- **seal(0) is an accepted, repeatable no-op** that still emits `SealStateChange`. `_state & sealState == 0` always holds for zero, so `AlreadyLocked` never fires; on an already-sealed pool it re-emits `_final = true`. Reject `_state == 0`.
- **Combined masks become unusable after any partial seal.** `seal(3)` after `seal(1)` reverts `AlreadyLocked` because the check rejects any overlap rather than any *new* bit. Consider ignoring already-set bits so `seal(7)` always means “seal everything”.
- **withdraw(token, 0) succeeds and emits a spurious Collect** while `withdraw(token)` correctly reverts `InsufficientFees` on a zero balance. The two overloads disagree on whether collecting nothing is an error.

I-5 try/catch cannot catch decode failures, so two named errors are unreachable

Severity	Informational
Target	src/OracleQuoter.sol L63-67, L78-88, L102-120, L152-162
Proof	test_POC_I5_codelessOracle_namedErrorUnreachable, test_POC_I5_codelessToken_namedErrorUnreachable

Description

`valueFor` and `getOracleRate` wrap every external call in `try/catch` so that a bad token or feed produces `TokenCallFailed` or `OracleCallFailed`. For the single most likely misconfiguration — an address with no code, such as an EOA pasted into `setOracle` — the wrappers never fire.

A high-level call to a codeless address *succeeds* at the EVM level and returns zero bytes. The failure happens afterwards, when Solidity cannot decode those zero bytes into the declared return tuple, and that decoding runs in the **caller's** frame, outside the region `try/catch` protects. Both PoCs confirm the revert data is empty (0x) rather than either named error, so a caller cannot distinguish “your oracle is not a contract” from any other failure. `setOracle` compounds this by validating only that the addresses are non-zero — it never probes the aggregator.

`RelativeQuoter` has the same unreachable error by a different mechanism: it declares `TokenCallFailed` and checks the success flag of a raw `.call`, but a codeless token makes that call *succeed* with empty data, so the failure again surfaces at `abi.decode` as a bare revert (`test_POC_relativeQuoter_codelessToken_namedErrorUnreachable`). `DecimalQuoter` declares `TokenCallFailed` and never references it at all.

This is the same class of problem as [I-2](#): the error surface documents attribution the code does not deliver. Across the codebase, five declared errors covering external-call failure are unreachable or unused — `SwapPool.TokenCallFailed`, `SwapPool.QuoterCallFailed`, `SwapPool.LimiterCallFailed`, `DecimalQuoter.TokenCallFailed`, and `RelativeQuoter.TokenCallFailed` for the codeless case.

Recommendation

Probe once at configuration time, where a mistake is cheap to surface:

```
function setOracle(address token, address oracleAddress) public onlyOwner {
    if (token == address(0) || oracleAddress == address(0)) revert InvalidToken();
```

```

    if (oracleAddress.code.length == 0 || token.code.length == 0) revert InvalidToken();
    IChainlinkAggregatorV3(oracleAddress).latestRoundData(); // reverts if not an aggregator
    IChainlinkAggregatorV3(oracleAddress).decimals();
    ...
}

```

I-6 `isActive()` is declared on both integration interfaces and never consulted

Severity	Informational
Target	src/interfaces/IFeePolicy.sol L9, src/interfaces/IProtocolFeeController.sol L9; src/SwapPool.sol L184-191, L228-235, L325-334
Proof	test_H2_isActive_neverConsulted (in AuditPoCSeal.t.sol)

Both interfaces advertise a kill switch that SwapPool never calls. A fee policy reporting `isActive() == false` still has its fees charged in full – the PoC has the policy return false and watches a 10 % fee accrue to `fees[]` regardless. This is harmless only by accident, because `FeePolicy.isActive()` is hard-coded to true; a future policy that honestly reports itself disabled would be ignored. `ProtocolFeeController` is safe for a different accidental reason: its `getProtocolFee()` already returns 0 when inactive, so the flag is redundant there.

Recommendation: either honour `isActive()` at the call sites or remove it from the interfaces. An interface method that consumers ignore is worse than no method, because implementers will assume it does something.

I-7 The protocol fee and the net output are unobservable from events

Severity	Informational
Target	src/SwapPool.sol L63-71, L308-317
Proof	test_H4_protocolFee_isUnobservable (in AuditPoCSeal.t.sol)

There is no event for the protocol-fee transfer, and `Swap.amountOut` carries the **gross** `quotedValue` rather than what the recipient received. In the PoC, Swap reports `amountOut = 100e18` and `fee = 1e18` while the user actually received 98.9e18 and the protocol took 0.1e18 – neither real figure appears in any log.

Off-chain accounting therefore cannot reconcile a swap without replaying the protocol fee calculation against the controller’s state at that block, which means indexers will silently misreport user proceeds and protocol revenue. `docs/SPEC.md` warns about the gross `amountOut`, which confirms it is known – but documenting a gap in observability does not close it.

Recommendation: emit the net amount and the protocol fee, either as extra Swap fields or as a companion `ProtocolFee(recipient, token, amount)` event. This pairs with the `_swap` return value proposed in H-1.

I-8 `AccountsIndex.remove()` reverts with the inverted error, and a test enshrines it

Severity	Informational
Target	src/AccountsIndex.sol L76; test/AccountsIndex.t.sol L121-125
Proof	test_AI_remove_absentAccount_revertsAlreadyExistsInsteadOfNotFound (in AuditPoCIndex.t.sol)

Removing an absent account reverts `AlreadyExists()`:

```

if (!this.have(_account)) revert AlreadyExists();

```

The semantics are exactly backwards. `AccountsIndex` declares `NotFound()` and uses it correctly in `activate`, `deactivate` and `time`; only `remove` reports the opposite condition. Any integrator branching on the error selector to distinguish “already removed” from “already present” gets the wrong answer.

Note for whoever fixes it: `test/AccountsIndex.t.sol:123` currently *expects* `AlreadyExists.selector` for the not-found case, so the test encodes the bug and will need updating alongside the contract. It illustrates a broader pattern –

error-name expectations in the suite were written from the implementation rather than checked against the declared error set.

Recommendation: `if (!this.have(_account)) revert NotFound();`

I-9 Registry hygiene: enumeration asymmetry and no token sanity check

Severity	Informational
Target	src/TokenUniqueSymbolIndex.sol L19, L54-66, L81-92, L144-146
Proof	test_TI_publicIdentifierListGetter_isOffByOneAgainstIdentifier, test_TI_nonTokenWithCatchAllFallback_isAcceptedAsAToken (in AuditPoCIndex.t.sol)

Two loose ends in TokenUniqueSymbolIndex:

- **identifierList is public alongside the 0-based identifier(idx), and the two are off by one.** identifierList(0) returns the bytes32(0) sentinel while identifier(0) returns the first real symbol. tokens is private, so the asymmetry is easy to trip over: an integrator reading the raw array gets a phantom leading element and a count one higher than identifierCount(). Make identifierList private — identifier(idx) and identifierCount() already cover it — or document the sentinel.
- **register() / add() perform no token sanity check.** The only requirement is that the address answers something ABI-decodable as bytes, so any contract with a permissive catch-all fallback is accepted as a “token” reporting whatever symbol it likes, and is then whitelisted by have() for SwapPool. This needs owner or writer, so it is defence-in-depth rather than a hole: the registry offers no protection against a writer being tricked into indexing a non-token. Probing decimals() or totalSupply() and requiring non-zero extcodesize would close it.

I-10 sweepERC721s skips the zero-recipient check on an empty array

Severity	Informational
Target	src/RescueVault.sol L81-85
Proof	test_POC_rescueVault_sweepERC721sEmptyArray_skipsZeroAddressCheck (in AuditPoCFunds.t.sol)

sweepERC721s is the only sweep that does not call _checkTo(to) itself; it relies on the inner sweepERC721 doing so inside the loop body. With an empty tokenId array the loop never runs, so `to == address(0)` is accepted and the call succeeds as a no-op — while `sweepETH(address(0))` and `sweepERC1155Batch(token, [], address(0))` both correctly revert ZeroAddress.

Harmless as written, but it is the kind of inconsistency that becomes a real hole the moment the loop body is refactored to call an unchecked internal helper.

Recommendation: hoist `_checkTo(to);` to the top of sweepERC721s, and have the loop call an internal `_sweepERC721` so the modifier and check are not re-run per item.

I-11 PeriodSimple’s default period == 0 is a one-second cooldown, not “no limit”

Severity	Informational
Target	src/PeriodSimple.sol L47-55
Proof	test_PS_zeroPeriodYieldsOneSecondCooldown; boundary pinned by test_PS_strictBoundaryMatchesSpec_holds (in AuditPoCToken.t.sol)

period defaults to 0 after initialize, and have() then evaluates `block.timestamp > lastUsed + 0`, so a subject can claim once per second — neither “always” nor “never”. For an operator who wires up a PeriodSimple but forgets setPeriod, the result looks configured while providing essentially no rate limit, which is the same failure direction as H-4.

The `>` rather than `>=` comparison makes the effective wait period + 1 seconds. That matches `SPEC.md:493` exactly and is therefore documented rather than a deviation.

Recommendation: reject `period == 0` in `setPeriod`, or require a non-zero period before the checker can be used.

I-12 CAT preference lists: duplicates, no opt-out, untraceable overrides, unvalidated entries

Severity	Informational
Target	<code>src/CAT.sol</code> L18, L41-43, L73-81
Proof	<code>test_CAT_duplicateTokensAreAccepted</code> , <code>test_CAT_accountCanNeverClearItsList</code> , <code>test_CAT_writerOverwritesUserPreferenceIndistinguishably</code> , <code>test_CAT_acceptsEoasAndNonErc20Addresses</code> , <code>test_CAT_hasNoSupportsInterface</code> (in <code>AuditPoCToken.t.sol</code>)

Five related gaps in the settlement-token preference registry:

- **Duplicates are accepted.** `_setTokens` validates length (1..5) and rejects zero addresses but never de-duplicates, so an account can spend its whole five-slot budget on one token and any consumer iterating the ordered list double-counts the same preference. An $O(n^2)$ check is trivially cheap at $n \leq 5$.
- **A list can never be cleared.** Zero length reverts `EmptyTokenList` and no delete entry point exists, so an account that once opted in can never opt out on-chain — not even a writer can clear it.
- **Writer overrides are indistinguishable from self-service updates.** `setTokensFor` lets a writer silently replace a user's own settlement ordering, and `TokensSet(account, tokens)` carries no operator field, so the two events are byte-identical. Off-chain consumers cannot tell who changed a user's preferences.
- **Entries are never validated.** EOAs, the registry itself and arbitrary non-ERC20 contracts are all accepted as "settlement tokens".
- **No supportsInterface.** CAT is the only contract in this group without it, breaking the ERC-165 discovery pattern its siblings advertise.

Checked non-finding: shrinking a list from five entries to two *does* correctly zero the dropped storage slots — verified at raw-storage level with `vm.load` — so there are no stale tail entries.

Recommendation: add the duplicate check; allow a zero-length list or add `clearTokens()`; add the operator to the event as `TokensSet(account, operator, tokens)`; optionally require `token.code.length > 0` or membership in `TokenUniqueSymbolIndex`; add `supportsInterface`.

I-13 Ceiling-add overflow in the reverse quote paths panics where the forward direction succeeds

Severity	Informational
Target	<code>src/SwapPool.sol</code> L273-274; <code>src/RelativeQuoter.sol</code> L124-126
Proof	<code>test_POC_I13_reverseNetToQuoted_ceilingAddOverflows</code> (in <code>AuditPoCGaps.t.sol</code>)

Description

Both reverse quote paths compute a ceiling division by adding the denominator before dividing, in plain 256-bit arithmetic:

```
// SwapPool._reverseNetToQuoted
return (_netOutput * ppmSquared + denominator - 1) / denominator;

// RelativeQuoter.reverseOutput
return (outputValue * outExchangeRate + inExchangeRate - 1) / inExchangeRate;
```

The product is evaluated before the division, so the intermediate overflows even when the final result would fit. `_netOutput * PPM^2` overflows for `_netOutput > ~1.16e65`; `outputValue * outExchangeRate` overflows at the same magnitude scaled by the rate. The PoC shows the asymmetry directly: with a 1 % pool fee, `getAmountOut` handles a `1.2e65` input and returns a normal quote, while `getAmountIn` for the same magnitude reverts with a bare `Panic()`

0x11) — the exact-output path fails where the exact-input path succeeds, with no named error to attribute it.

This is unreachable for any realistic token supply (it needs $1e47$ whole tokens at 18 decimals), so it is a robustness note rather than an exploitable one — the same class as L-4, one function further down the same call chain. It is recorded because the ceiling-add pattern silently defeats the widening that `FixedPointMathLib.fullMulDivUp` exists to provide, and both sites are already being touched for M-1 and M-8.

Recommendation

Use `FixedPointMathLib.fullMulDivUp(_netOutput, ppmSquared, denominator)` and `fullMulDivUp(outputValue, outExchangeRate, inExchangeRate)`, which carry the intermediate at 512 bits and only fail when the *result* does not fit.

I-14 A GiftableToken held by a SwapPool is permanently locked at expiry, and its frozen balance keeps counting toward liquidity

Severity	Informational
Target	composition seam — <code>src/GiftableToken.sol</code> L110-112 × <code>src/SwapPool.sol</code> L292-298, L336-375
Proof	<code>test_POC_I14_expiredVoucher_permanentlyLockedInPool</code> , <code>test_POC_I14_expiredVoucher_swapRevertsAgainstFrozenBalance</code> (in <code>AuditPoCGaps.t.sol</code>)

Description

The protocol's own voucher token expires, and from that moment every transfer reverts `TokenExpired`. A pool holding that token as inventory inherits the freeze three ways, none of which has a remedy:

1. **withdrawLiquidity reverts.** The owner's emergency lever routes through `IERC20.transfer`, which hits the expiry hook. The dead inventory can never be cleared from the pool — not by the owner, not by anyone.
2. **Accrued fees in the expired token can never be collected.** `withdraw(token)` reverts the same way, so any `fees[voucher]` balance at expiry is stranded permanently.
3. **The frozen balance still counts as available liquidity.** In decoupled mode the check is `balance - fees[token] >= quotedValue`, and the frozen balance satisfies it — so a swap quoting the expired token as `tokenOut` passes pricing and liquidity, reaches the payout transfer, and only then reverts `TokenExpired`. The pair looks liquid to any integrator reading the quote path until the moment of execution.

The locked tokens are zero-value by design (that is what expiry means), so there is no direct loss — the finding is the permanent operational dead-end: no function can remove the dead balance, `balanceOf` keeps reporting it, and the fee reserve is uncollectable. M-20 covers the same freeze from the token side (burn is blocked); this is the pool-side consequence, found at the composition seam the first pass flagged as under-explored.

Recommendation

Document that expired-voucher balances in pools are unrecoverable by design, and give operators a runbook step: de-list the token from the pool's `tokenRegistry` *before* expiry (bearing in mind M-3 — de-listing stops deposits, not extraction attempts). If pool-side cleanup is ever wanted, `withdrawLiquidity` needs a variant that tolerates a reverting transfer for expired tokens; otherwise accept the lock as the cost of expiry and state it in `docs/SPEC.md`.

I-15 withdrawLiquidity has no zero-address check on to

Severity	Informational
Target	<code>src/SwapPool.sol</code> L367-375
Proof	<code>test_POC_I15_withdrawLiquidity_toZeroAddress_burnsTokens</code> (in <code>AuditPoCGaps.t.sol</code>)

Description

The four-argument swap `withdraw` validates its recipient (`InvalidRecipient`), but the owner's emergency withdrawal

validates nothing:

```
function withdrawLiquidity(address token, address to, uint256 amount) external onlyOwner returns (uint256) {
    uint256 balance = IERC20(token).balanceOf(address(this));
    if (amount > balance) revert InsufficientBalance();
    bool success = IERC20(token).transfer(to, amount);
    ...
}
```

`withdrawLiquidity(token, address(0), amount)` succeeds against solady-style tokens that do not check the recipient (see L-21) — the assets are **burned, not recovered**. An operator typo (an empty `--to` variable in a shell script) destroys pool funds irreversibly instead of rescuing them. Owner-gated, hence Informational; `RescueVault._checkTo` in the same repository shows the intended pattern, and the pool already has the `InvalidRecipient` error declared and in use one function away.

Recommendation

```
if (to == address(0)) revert InvalidRecipient();
```

I-16 Deployment tooling: the private key travels on the command line, and over-long registry identifiers are silently truncated

Severity	Informational
Target	<code>cmd/ge-publish/main.go</code> L174; <code>docs/DEPLOY.md</code> L17, L23; <code>pkg/publish/contracts/*/contractregistry.go</code> , <code>tokenuniquesymbolindex.go</code> (<code>toBytes32Slice</code>)
Proof	code trace

Description

Two operational gaps in the deployment path, neither of which touches contract logic but both of which decide what ends up on-chain.

1. The deployer key is exposed to the process list and shell history. `--private-key` is a CLI flag, and `docs/DEPLOY.md` instructs the operator to export `PRIVATE_KEY=<deployer hex key>` and then expand it into `$BASE`, so the key appears in the shell's history file and in `ps` output for every command in the runbook. These are mainnet deployment credentials; anything later running on the same machine (or anyone reading `.bash_history`) holds them. Environment variables are only marginally better; the runbook normalises the flag form.

2. `toBytes32Slice` left-crops inputs longer than 32 bytes instead of rejecting them. `common.RightPadBytes(value, 32)` returns an over-long input unchanged, and `common.BytesToHash` then crops **from the left**, keeping the last 32 bytes. Two `--registry-identifiers` (or `--token-index-symbols`) longer than 32 bytes that differ only in their leading bytes collapse to the same `bytes32` key — producing the duplicate-identifier corruption of L-15 or an unintended symbol key in `TokenUniqueSymbolIndex`, silently, at deploy time. The on-chain `TokenSymbolTooLong` guard does not help because `initialize` takes the keys directly and never measures the original strings.

Recommendation

Read the key from a file or keystore (or prompt), never a flag, and drop the `export PRIVATE_KEY=...` line from the runbook. Reject `len(value) > 32` in `toBytes32Slice` with an explicit error naming the offending entry.

Method, surface coverage, and what was not done

External surface

src/ exposes **157 external or public functions** across 17 contracts. All 157 were read. 152 are either named in a finding above or called by a test in test/audit/; the remaining five — EthFaucet.tokenAmount, FeePolicy.getDefaultFee, ProtocolFeeController.getProtocolFeeRecipient, ProtocolFeeController.setProtocolFeeRecipient and OracleQuoter.removeOracle — are covered by behavioural assertions in AuditPoCInvariant.t.sol so the surface is verified rather than assumed. All five behave as specified; removeOracle fails closed with OracleNotSet, and setProtocolFeeRecipient is owner-gated and rejects the zero address.

Exploit classes swept

Class	Where it was checked	Outcome
Reentrancy	SwapPool, Splitter, RescueVault, EthFaucet, TokenUniqueSymbolIndex	M-2 found; the other four are safe, each for a different reason — see the <code>_holds</code> tests
Integer overflow / underflow / truncation	every arithmetic site and every unchecked block	H-3, M-5, L-4, L-11, L-22
Access control	all 157 functions	every state-changing function is guarded; no gaps
Unchecked return values, unsafe low-level calls	all six raw <code>.call</code> sites	L-8, L-13, M-18, I-5
Denial of service / griefing	push-payment loops, unbounded arrays, caps	M-7, M-13, M-16, L-16, L-20, L-24
Transaction ordering / front-running	swap execution, split updates, distribution granularity	H-1, M-14, M-15
Hash collision (abi.encodePacked)	Splitter, getHash, FeePolicy pair keys	L-17 real but unreachable; FeePolicy sound
Signature replay / malleability	GiftableToken EIP-2612 (the only signature path)	sound — domain separator, chain id and per-owner nonce all correct
Timestamp dependence	oracle staleness, cooldowns, expiry	L-3, L-22, L-25, I-11
Proxy / upgrade / storage layout	all 14 proxied contracts	H-2, L-6, L-7
Weak randomness	—	not applicable; no randomness anywhere in <code>src/</code>

Stateful invariant fuzzing

`AuditPoCInvariant.t.sol` runs a bounded action surface — swaps in both directions, bare deposits, full and partial fee collection, and live fee-rate changes — against a `feesDecoupled` pool, and asserts `fees[token] <= balanceOf(pool, token)` after every step.

The configuration deliberately **excludes the three known causes** of that property breaking: `withdrawLiquidity` is not in the handler (M-6), the token has no transfer hook (M-2), and the pool is decoupled rather than coupled (L-2). **256 runs, 128 000 calls, zero failures.** That is a useful negative result: within this action surface those three causes appear to be exhaustive, so fixing them should restore the invariant rather than merely postponing it.

Not done

The following were not done, so the report is not read as more complete than it is:

- **Invariant campaigns were run on SwapPool only.** Splitter (balance conservation under adversarial allocations), the two enumerable registries (have/entry consistency across long mutation sequences) and GiftableToken (supply accounting) would each benefit, and are the highest-value next step.
- **No fully wired end-to-end system test.** Contracts were audited individually plus their direct counterparties. [M-9](#) was found at a composition seam — AccountsIndex used as a SwapPool/EthFaucet registry — which is evidence that more may sit at seams no single-contract review reaches.
- **No symbolic execution or formal verification.** The repository contains a `mythril.json` config that was not exercised. Halmos or Mythril over the arithmetic in `_reverseNetToQuoted` and `_scaleAmountByPercentage` would be a reasonable complement to the exhaustive-case testing done here.
- **The ERC20 approve race is not reported.** GiftableToken inherits solady's approve with no `increaseAllowance/decreaseAllowance`, so the classic approve-front-running pattern is present. It is omitted deliberately: it is inherent to the ERC20 standard, universally present, and handled by wallet tooling rather than by token contracts.
- **Symbol-squatting was analysed as a state-corruption bug, not as a race.** [M-11](#) covers the corruption; front-running a legitimate registration to claim a ticker first is a related angle that was not tested.
- **Economic and governance modelling is out of scope.** Whether the protocol fee floor, the multiplier band or the split percentages make sense as policy is not assessed — only whether the code does what the specification says.

Follow-up pass (2026-08-10)

A second, narrower sweep ran after the 67 findings above were written up, aimed at the gaps the first pass declared. It produced eight findings — [L-28](#) ... [L-31](#) and [I-13](#) ... [I-16](#) — all reproduced in `test/audit/AuditPoCGaps.t.sol`:

- **Composition seams** (called out above as under-explored): GiftableToken expiry × SwapPool inventory was traced end-to-end — [I-14](#).
- **Reentrancy beyond the M-2 path:** the inbound (`tokenIn`) side of the swap, where the limiter check precedes the transfer — [L-29](#).
- **Input validation asymmetries:** functions that validate an address in one place and not its sibling — `giveTo` ([L-28](#)), `withdrawLiquidity` ([I-15](#)).
- **Reverse/forward symmetry:** the ceiling-add pattern in the reverse quote paths — [I-13](#).
- **The deployment tooling itself** (`cmd/ge-publish`, `pkg/publish`, `docs/DEPLOY.md`): a dead command path ([L-30](#)), a one-per-name proxy salt ([L-31](#)), key handling and silent identifier truncation ([I-16](#)).

Still not done: invariant campaigns on Splitter, the registries and GiftableToken; symbolic execution; and the economic/governance modelling. Those remain the highest-value next steps.

Test coverage gaps

The existing suite is 379 passing tests and is thorough on the happy path, fee arithmetic, seal transitions and access control. Every finding above survived it. Measured coverage explains why in one line: **line coverage is high and branch coverage is not**, so the suite executes almost every statement while exercising only one side of most decisions.

forge coverage --no-match-path 'test/audit/*', contracts sorted by branch coverage:

Contract	% Lines	% Branches	% Funcs
RelativeQuoter	62.12	47.06	75.00
DecimalQuoter	41.67	50.00	33.33
Splitter	96.67	50.00	92.31
EthFaucet	83.12	55.00	88.24
TokenUniqueSymbolIndex	100.00	60.00	100.00
OracleQuoter	83.33	62.50	100.00
RescueVault	92.00	66.67	85.71
SwapPool	99.38	67.44	100.00
AccountsIndex	100.00	84.62	100.00
CAT, ContractRegistry, FeePolicy, Limiter, PeriodSimple, ProtocolFeeController, SwapRouter	100.00	100.00	100.00
Total	73.20	61.50	66.30

Two rows stand out. SwapPool sits at 99.38 % of lines and 67.44 % of branches — the single most complex contract in the codebase reads as almost fully covered by the metric most people quote, while a third of its decisions have never been taken in both directions. And RelativeQuoter and DecimalQuoter, the two quoters that produce [M-8](#) and [L-5](#), are the two least covered contracts in the repository despite being directly in the pricing path.

For reference, the audit tests move SwapPool branches to 76.74 %, RelativeQuoter to 64.71 % and DecimalQuoter to 83.33 %. **Branch coverage is the metric to gate on**, and nothing here should be read as a target to hit by adding assertions — the specific gaps below are what matter.

No adversarial ERC20 anywhere. All four token mocks (MockERC20, MockERC20R, MockToken) are well-behaved. There is no fee-on-transfer token, no token that returns false, no token that returns no data at all (`_deposit` would revert on ABI decoding), no rebasing token, and no token with a transfer hook. C-2 and M-2 both live in that gap. Add a shared `test/mocks/` family of hostile tokens and run the pool's swap paths against each.

No reentrancy test. Nothing in the suite re-enters SwapPool. Given the contract has no `nonReentrant` guard and is designed to hold third-party community tokens, a reentrancy harness is the single highest-value addition.

No tokenIn == tokenOut case. Not exercised on any pool or any quoter. This is C-1's simplest form.

No ungated-pool test. `tokenRegistry == address(0)` and `tokenLimiter == address(0)` are documented configurations that no test covers. `test_getQuote_no_quoter` covers the third optional module but not these two.

Quoter round-trip fuzzing only covers the exact case. `test_reverseValueFor_roundtrip_fuzz` and `test_reverseValueFor_roundtrip_withMultiplier_fuzz` use `SRF↔MBAO`: both 6 decimals, both 8-decimal feeds, prices `1e8` and `5e7`. The rate ratio is exactly 2, so the intermediate division is lossless and M-1 cannot appear. Needed: a fuzz over both decimal directions (`6→18` and `18→6`), with feed decimals 8 and 18 mixed, non-round prices, and the multiplier at its range boundaries. `test_getAmountIn_roundtrip_fuzz` has the same blind spot from the other side — MockQuoter multiplies by an integer rate and reverses with exact ceiling division, so it is never lossy. `test_getAmountIn_with_quoter` (`test/SwapPool.t.sol:480`) contains no assertions at all: it sets up rates and then abandons the check with a comment saying the mock cannot express the inverse. That is the exact case M-1 lives in, and it is the one case the suite declines to test.

No dust or zero-output amounts. Every SwapPool test swaps whole-token amounts. Nothing swaps 1 wei, and nothing swaps an amount small enough for the quoter to truncate it to zero, which is how L-5 goes unnoticed. Add boundary cases at 1, at `10 ** (inDecimals - outDecimals) - 1`, and at the pool's exact available liquidity.

Fee boundaries untested. The highest pool fee anywhere in the suite is 5 % (`test_getFee`), while FeePolicy permits

100 %. Nothing probes the 909 090 PPM cliff (M-5), and no test combines a large pool fee with a large protocol fee.

Accounting invariants never asserted. No test checks `fees[token] <= balanceOf(pool, token)`, which is the invariant M-2 and L-2 both break. An invariant test over a random sequence of deposits, swaps, fee collections and `withdrawLiquidity` calls would catch both. `withdrawLiquidity` is also never tested against a pool with accrued fees.

Zero-feeAddress behaviour half-covered. `test_collectFees_revertIf_invalid_fee_address` covers the collection revert but nothing checks what a *swap* does in that configuration (L-1).

Router paths. Only 1-hop and 2-hop well-formed paths are tested. Missing: a discontinuous path, a path longer than two hops, a path that revisits the same pool, and a path where one pool lacks liquidity.

Oracle edge cases. `test_valueFor_revertIf_oracleReturnsZero` covers `answer == 0` but not a negative answer, `updatedAt == 0`, or a future `updatedAt`. `MockChainlinkAggregator` has no way to set `updatedAt` independently of `block.timestamp`, which is why the whole class is untested; adding a setter is the enabling change. `setMaxStaleness` is tested only with 3600 — neither 0 nor `type(uint256).max` is exercised, and no test uses two feeds with different heartbeats, which is what M-4 needs.

No storage-layout or upgrade tests. `multiplier`'s zero fallback is tested (`test_multiplier_defaultIsZero`) but nothing pins the slot assignments, so the `maxStaleness` gap in L-6 is invisible. A test that asserts each appended variable's slot index, and one that zeroes each slot to simulate an in-place upgrade, would have caught it and would protect future appends. The same omission hides two of the four seal defeats in H-2: nothing in the suite deploys behind a real `ERC1967Factory`, upgrades an implementation, or asserts that `sealState` survives. `test/audit/AuditPoCSeal.t.sol` now does all three and those tests are worth keeping as permanent upgrade-safety regressions.

****Nothing tests the pool against a *sealed* configuration.**** The seal tests verify that the three covered setters revert, and stop there. No test asks what remains mutable afterwards, which is why H-2's first defect — swapping the registry out from under a fully sealed pool — went unnoticed. A test that seals a pool and then asserts the complete set of still-reachable state transitions would be the single most valuable addition on the access-control side.

Fee collection is never tested against `withdrawLiquidity`. No test runs the two against each other, so the permanent `fees[] > balance` brick in M-6 is invisible. The general fix is an invariant test: `assert fees[token] <= balanceOf(pool, token)` after every operation in a random sequence of deposits, swaps, fee collections and liquidity withdrawals. That one invariant covers M-2, M-6 and L-2 at once.

deposit is only ever called by the party being credited. Every test deposits on its own behalf, so nothing exercises a third party depositing to manipulate pool state — the mechanism behind M-7. Add a grieving test where an unrelated address saturates a limiter cap and an honest swap is then attempted.

Zero-address arguments to `initialize`. No test passes `owner == address(0)` to either `SwapPool.initialize` or `OracleQuoter.initialize` (L-7). `OracleQuoter`'s zero-baseCurrency case is tested, which shows the omission is an oversight rather than a decision.

Implementation contract not tested directly. `test_initialize_revertIf_already_initialized` targets the clone. Nothing asserts that `initialize` on the implementation itself reverts, which is what `_disableInitializers()` in the constructor exists to guarantee.

Splitter and RescueVault

Four of Splitter's five validation errors have no test at all. `InvalidAllocationsSum`, `TooFewAccounts`, `AccountsAndAllocationsMismatch` and `AllocationMustBePositive` are never exercised; the only negative tests are `DuplicateAccount` and `InvalidHash`. Every allocation used anywhere in the suite is a plausible PPM value between 30 000 and 600 000, so nothing comes within nine orders of magnitude of the `uint32` wrap in H-3. One negative test per declared error, plus a fuzz over allocation values, catches it immediately.

No recipient in the Splitter suite is a contract. All are bare addresses (`0x1001`, `0x2000+i`), so a reverting `receive()`, a gas-burning `receive()`, a contract with no `receive()`, and reentrancy are all untested — M-13 and the whole reentrancy question go unexercised. Nor does any test pass `address(0)` or the splitter itself as a recipient (M-12); the suite's `MockERC20` has no zero-address check either, so even if one were used the ERC20 brick would be masked by the mock.

Every Splitter test distributes exactly once, from a balance of at least 1e6 wei. `test_distributeETH_remainderGoesToLastRecipient` uses `1 ether + 1`, where flooring is benign. The case that matters is a balance below `1_000_000 / p_min`, where leading shares floor to zero — and there is no repeated-distribution test at all, so the drip capture in M-14 cannot appear. Recipient counts also stop at 10 with no gas assertion, hiding the quadratic cliff in L-16.

The hash commitment is only tested on the happy path, and the test mirrors the implementation. `test_initialize_setsHashAndOwner` recomputes `keccak256(abi.encodePacked(...))` in the test rather than challenging it, and no test supplies mismatched array lengths. The only thing that makes L-17 unexploitable is `_validateSplit` running before `_validateHash`; swapping those two lines would break nothing in CI.

RescueVault's test suite asserts the hazard as a feature. `test_receiveAndFallback_acceptETH` sends `hex"deadbeef"` with value and requires success, pinning L-18 in as desired behaviour instead of asking what a third party calling a real selector on a squatted address should see. Separately, `test_revertIf_notAdmin` and `test_revertIf_zeroRecipient` each cover only sweepETH — the other six sweeps have no access-control or zero-recipient test, which is exactly how the missing `_checkTo` in I-10 survived. Every batch test uses distinct, well-formed, vault-owned elements, so all three parts of L-20 pass unnoticed. And three of the four NFT tests mint directly to the vault rather than pushing through `safeTransferFrom`, so `onERC1155Received` and `onERC1155BatchReceived` are never actually invoked by a transfer.

Registries

One token shape, and it is always well-formed. `test/TokenUniqueSymbolIndex.t.sol` uses a single `MockToken` with 3-character string symbols. There is no empty-symbol token (M-10), no symbol-changing token (M-11), no `bytes32`-returning token (L-13), and no non-token contract. A registry whose whole job is reading a value from an untrusted external contract needs a matrix of adversarial mock shapes, including the degenerate `"` case.

No test asserts the whitelist/enumeration invariant. `test_remove` and `test_remove_swap_last` check `entry()` and `have()` separately but never that `have(entry(i))` holds for all `i`, nor that `addressOf(identifier(i)) == entry(i)`. M-10 and M-11 both manifest *solely* as a violation of that invariant, so one property assertion after every mutation would have caught both. Relatedly, `test_register_revertIf_symbol_exists` covers duplicate *symbols* with two distinct addresses — the case that is guarded — and never the duplicate *address* case that is not.

Fuzz exclusions hide the inputs that matter. `AccountsIndex`'s two fuzz tests both `vm.assume(account != address(0))`, which is precisely the input L-12 needs. `ContractRegistry`'s fuzzer generates identifiers as `keccak256(abi.encodePacked(i))`, collision-free by construction, so the duplicate path in L-15 is structurally unreachable. Fuzz exclusions should be justified against the contract's actual guards rather than assumed.

time() is only ever read for an active account. `test_time` and `test_fuzz_time` read it immediately after `add`. `test_multiple_activation_cycles` performs the exact deactivate/activate sequence that triggers L-11 but never inspects the timestamp — one assertion inside a test that already exists.

No test exercises have() from a consumer's perspective, which is why M-9 is invisible: the suite checks `have()` and `isActive()` in isolation but never the relationship the consumers depend on. A cross-contract test wiring `AccountsIndex` into `SwapPool.tokenRegistry` and `EthFaucet.registry`, then asserting a deactivated entry is rejected, would close it.

Faucet, token and cooldown

The faucet suite never claims twice from the same address. `test_gimme` and `test_giveTo` each make a single call, and `test_check_with_period_checker` attaches a `PeriodSimple` with `period == 0` but never claims through it. No test anywhere configures a non-zero period. A claim loop is one for statement, and it is the difference between the suite passing and H-4 / M-17 being obvious.

Sealing is tested only for its revert behaviour, never for its consequences. `test_setAmount_revertIf_sealed`, `test_setRegistry_revertIf_sealed` and `test_setPeriodChecker_revertIf_sealed` each seal a field that is still `address(0)` and then assert only that the setter reverts `Sealed`. Not one goes on to attempt a claim — which is exactly the assertion that would have surfaced H-5, the permanently open faucet. Testing that a guard fires is not the same as testing what the guard leaves behind.

amount == 0 is explicitly excluded. `test_fuzz_setAmount` and `test_fuzz_gimme` both `vm.assume(amount > 0)` and no unit test uses zero (L-24). `nextTime` and `nextBalance` have no coverage at all (L-26), and nothing asserts anything about ETH egress or a withdraw path (M-16). `MockRegistry` is always ABI-perfect, so no test points a gate at an EOA or a short-returning contract (M-18).

GiftableToken never reads expired() after a reverted transfer. `test_transfer_revertIf_expired` and `test_transferFrom_revertIf_expired` assert only the revert, while `test_applyExpiry_expired` sets the flag through a *direct* `applyExpiry()` call. The two paths are never cross-checked, which is precisely the gap M-19 lives in — one `assertFalse(token.expired())` after an existing revert assertion would have caught it. No test burns or mints after expiry (M-20), and none uses `address(0)` as a target (L-21).

approve, allowance, permit, DOMAIN_SEPARATOR and nonces are untested. approve appears only as a setup step, with no allowance assertion anywhere and no permit test at all, so the inherited Permit2 allowance override (M-21) was never observed. When a contract inherits a substantial base implementation, the inherited surface needs its own tests — the absence of custom code is not the absence of behaviour.

PeriodSimple's fuzz bounds exclude the failure region. test_fuzz_setPeriod and test_fuzz_poke both assume `period < 365` days, and test_edge_case_large_threshold stresses the *threshold* to `type(uint256).max` but never the *period* — so the overflow in L-22 is unreachable by construction. test_poke_by_owner proves the owner can poke but never checks the consequence (L-23), and test_next_first_use checks `next() == 0` only while `period == 0`, hiding L-25.

CAT's only replacement test shrinks 2 → 1 through the getters. It never goes 5 → 2 and never inspects raw storage, so the tail-clearing property was assumed rather than verified — it does in fact hold, but that was luck rather than evidence. There is no test for duplicates, for clearing a list, or for non-contract entries (I-12).